Classic LLM Architectures

A Detailed Technical History of Every Major Model

Papers, Innovations, and Design Decisions in Chronological Order

May 2026

Contents

1 The GPT Family: Autoregressive Language Models **5**

1.1 GPT-1: "Improving Language Understanding by Generative Pre-Training" (Radford et al., 2018) . . . 5

1.1.1 The Paper and Problem . . . 5

1.1.2 Architecture . . . 5

1.1.3 Training . . . 5

1.1.4 Fine-tuning Approach (The "GPT Paradigm") . . . 6

1.1.5 Novelty and Impact . . . 6

1.2 GPT-2: "Language Models are Unsupervised Multitask Learners" (Radford et al., 2019) . . . 6

1.2.1 The Paper and Problem . . . 6

1.2.2 Architectural Evolution . . . 6

1.2.3 Zero-Shot Transfer Paradigm . . . 6

1.2.4 Why GPT-2 was Initially Withheld . . . 7

1.3 GPT-3: "Language Models are Few-Shot Learners" (Brown et al., 2020) . . . 7

1.3.1 The Paper . . . 7

1.3.2 Architecture Scaling . . . 7

1.3.3 The "In-Context Learning" Discovery . . . 7

1.3.4 Key Results . . . 7

1.3.5 Novelty Summary . . . 8

1.4 GPT-3.5, InstructGPT, and ChatGPT . . . 8

1.4.1 InstructGPT (Ouyang et al., 2022) . . . 8

1.4.2 ChatGPT (November 2022) . . . 8

1.5 GPT-4: "GPT-4 Technical Report" (OpenAI, 2023) . . . 8

1.5.1 The Paper . . . 8

1.5.2 What We Know . . . 8

1.5.3 Predictable Scaling (Key Innovation) . . . 9

1.5.4 Key Results . . . 9

1.6 GPT-4o: Omni-Modal Model (OpenAI, May 2024) . . . 9

1.6.1 The Model . . . 9

1.6.2 Architectural Innovations . . . 9

1.6.3 The Tokenizer Innovation . . . 10

1.6.4 o1 and o3: Reasoning Models . . . 10

1.7 Summary: The GPT Architectural Lineage . . . 10

2 The BERT Family: Bidirectional Encoder Models **11**

2.1 BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2019) . . . 11

2.1.1 The Paper . . . 11
2.1.2 Architecture . . . 11
2.1.3 Pre-Training Objectives (The Dual Objective Innovation) . . . 11
2.1.4 Fine-Tuning . . . 12
2.1.5 Results and Impact . . . 12
2.2 RoBERTa: A Robustly Optimized BERT Approach (Liu et al., 2019) . . . 12
2.2.1 The Paper . . . 12
2.2.2 Changes from BERT . . . 12
2.2.3 Results . . . 13
2.3 ALBERT: A Lite BERT (Lan et al., 2019) . . . 13
2.3.1 The Paper . . . 13
2.3.2 Key Innovations . . . 13
2.3.3 Results . . . 13
2.4 DistilBERT: Distilled BERT (Sanh et al., 2019) . . . 14
2.4.1 The Paper . . . 14
2.4.2 Architecture . . . 14
2.5 DeBERTa: Decoding-enhanced BERT with Disentangled Attention (He et al., 2021) . . . 14
2.5.1 The Paper . . . 14
2.5.2 Key Innovation: Disentangled Attention . . . 14
2.5.3 Enhanced Mask Decoder (EMD) . . . 14
2.5.4 DeBERTaV3 (He et al., 2023) . . . 15
2.5.5 Summary: BERT Architectural Contributions . . . 15

3 T5, PaLM, and Google's Training Breakthroughs 17
3.1 T5: Text-to-Text Transfer Transformer (Raffel et al., 2020) . . . 17
3.1.1 The Paper . . . 17
3.1.2 Architecture . . . 17
3.1.3 The "Colossal Clean Crawled Corpus" (C4) . . . 17
3.1.4 Span Corruption Pre-Training . . . 18
3.1.5 Key Findings (The Ablation Study) . . . 18
3.2 PaLM: Pathways Language Model (Chowdhery et al., 2022) . . . 18
3.2.1 The Paper . . . 18
3.2.2 Architecture (PaLM's Innovations) . . . 18
3.2.3 Training System: Pathways . . . 19
3.2.4 Results . . . 19
3.2.5 Variants . . . 19
3.3 Chinchilla: Training Compute-Optimal LLMs (Hoffmann et al., 2022) . . . 19
3.3.1 The Paper . . . 19
3.3.2 The Chinchilla Scaling Law . . . 19
3.3.3 Chinchilla Model . . . 20
3.3.4 The Key Insight . . . 20
3.3.5 Later Revision: Chinchilla is Not a Ceiling . . . 20
3.4 Gemini: Google's Multimodal Frontier (2023–2024) . . . 20
3.4.1 Gemini 1.0 (December 2023) . . . 20
3.4.2 Gemini 1.5 (February 2024) . . . 20
3.4.3 Gemini 2.0 (December 2024–2025) . . . 21

4 The LLaMA Family: Democratizing Open-Weight LLMs **23**
4.1 LLaMA 1: Open and Efficient Foundation Language Models (Touvron et al., 2023) . . . 23
4.1.1 The Paper . . . 23
4.1.2 Architectural Innovations . . . 23
4.1.3 Training . . . 24
4.1.4 Results . . . 24
4.2 LLaMA 2: Open Foundation and Fine-Tuned Chat Models (Touvron et al., 2023) . . . 24
4.2.1 The Paper . . . 24
4.2.2 Architectural Changes from LLaMA 1 . . . 24
4.2.3 Chat Model Training (The Alignment Pipeline) . . . 24
4.2.4 Results . . . 25
4.3 LLaMA 3 and 3.1: The Big Leap (Meta, April–July 2024) . . . 25
4.3.1 LLaMA 3 (April 2024) — The Paper . . . 25
4.3.2 Architectural Changes from LLaMA 2 . . . 25
4.3.3 LLaMA 3.1 (July 2024) . . . 26
4.3.4 Training Recipe (LLaMA 3) . . . 26
4.3.5 LLaMA 4 (2025) . . . 26
4.4 LLaMA Architectural Evolution Summary . . . 27

5 Mistral: Efficient Open-Weight Models from Europe **29**
5.1 Mistral 7B (Jiang et al., 2023) . . . 29
5.1.1 The Paper . . . 29
5.1.2 Architectural Innovations . . . 29
5.1.3 Training . . . 29
5.1.4 Mistral 7B Instruct . . . 30
5.2 Mixtral 8×7B: Sparse Mixture of Experts (Jiang et al., 2024) . . . 30
5.2.1 The Paper . . . 30
5.2.2 Architecture . . . 30
5.2.3 Key Design Decisions . . . 30
5.2.4 Results . . . 30
5.3 Mistral Large, Nemo, and Codestral . . . 30
5.3.1 Mistral Large (2024) . . . 30
5.3.2 Mistral Nemo 12B (July 2024) . . . 31
5.3.3 Codestral 22B (May 2024) . . . 31
5.3.4 Pixtral 12B (September 2024) . . . 31
5.3.5 Mistral Small 3 (2025) . . . 31
5.4 Mistral's Design Philosophy . . . 31
5.5 Mistral Family Summary . . . 32

6 Qwen Series: Alibaba's Multilingual Powerhouse **33**
6.1 Qwen (Tongyi Qianwen) 1.0 (September 2023) . . . 33
6.1.1 The Paper . . . 33
6.1.2 Architecture . . . 33
6.1.3 Training . . . 33
6.1.4 Qwen-Chat . . . 34
6.2 Qwen 1.5 (February 2024) . . . 34

6.2.1 Changes from 1.0 . . . 34
6.3 Qwen2 (June 2024) . . . 34
6.3.1 The Paper . . . 34
6.3.2 Architectural Refinements . . . 34
6.3.3 Qwen2-57B-A14B (MoE) . . . 34
6.3.4 Results . . . 34
6.4 Qwen2.5 (September 2024) . . . 35
6.4.1 Key Improvements . . . 35
6.4.2 Results . . . 35
6.5 Qwen2.5-VL and Qwen-Audio . . . 35
6.6 Qwen Family Summary . . . 35

7 Gemma and Phi: Small But Mighty **37**
7.1 Gemma: Google's Open Model Family (February 2024) . . . 37
7.1.1 The Paper . . . 37
7.1.2 Architecture . . . 37
7.1.3 Training . . . 37
7.1.4 Gemma 2 (June 2024) . . . 38
7.1.5 Results . . . 38
7.1.6 RecurrentGemma (April 2024) . . . 38
7.1.7 CodeGemma and ShieldGemma (2024) . . . 38
7.2 Phi Series: Data Quality Over Scale (Microsoft) . . . 38
7.2.1 Phi-1: "Textbooks Are All You Need" (Gunasekar et al., June 2023) 38
7.2.2 Phi-1.5 (September 2023) . . . 39
7.2.3 Phi-2 (December 2023) . . . 39
7.2.4 Phi-3 (April–August 2024) . . . 39
7.2.5 Phi-3-Vision . . . 39
7.2.6 Phi-3.5 (August 2024) . . . 40
7.2.7 Phi-4 (December 2024) . . . 40
7.2.8 The Phi Philosophy Summary . . . 40
7.3 Small Model Comparison . . . 40

8 The DeepSeek Family: A Complete Technical History **41**
8.1 Paper #1: DeepSeek LLM—Scaling Open-Source Language Models (Jan 2024) . . . 41
8.1.1 The Paper . . . 41
8.1.2 Scale and Architecture . . . 41
8.1.3 Key Innovation: Scaling Laws for Data . . . 41
8.1.4 Results . . . 42
8.2 Paper #2: DeepSeekMoE—Towards Ultimate Expert Specialization (Jan 2024) . . . 42
8.2.1 The Paper . . . 42
8.2.2 DeepSeekMoE Architecture . . . 42
8.2.3 Fine-Grained Expert Segmentation . . . 42
8.2.4 Load Balancing . . . 43
8.2.5 Key Results . . . 43
8.3 Paper #3: DeepSeek-V2—Strong, Economical, and Efficient (May 2024) 43
8.3.1 The Paper . . . 43

8.3.2 Innovation 1: Multi-Head Latent Attention (MLA) . . . 43
8.3.3 Innovation 2: DeepSeekMoE at Scale . . . 44
8.3.4 Training and Cost . . . 44
8.3.5 Results . . . 44
8.4 Paper #4: DeepSeek-Coder-V2 (June 2024) . . . 44
8.4.1 The Paper . . . 44
8.4.2 Architecture . . . 44
8.4.3 Results . . . 45
8.5 Paper #5: DeepSeek-V3—The Giant Leap (December 2024) . . . 45
8.5.1 The Paper . . . 45
8.5.2 Architectural Innovations . . . 45
8.5.3 Training Details . . . 46
8.5.4 Alignment (Post-Training) . . . 46
8.5.5 Results . . . 46
8.6 Paper #6: DeepSeek-R1—Reasoning via Reinforcement Learning (January 2025) . . . 47
8.6.1 The Paper . . . 47
8.6.2 DeepSeek-R1-Zero: Pure RL, No SFT . . . 47
8.6.3 DeepSeek-R1: Cold-Start Data + RL . . . 47
8.6.4 Distillation to Smaller Models . . . 48
8.6.5 Key Results . . . 48
8.7 Paper #7: DeepSeek-Prover (2024–2025) . . . 48
8.8 Paper #8: DeepSeek-VL/VL2 (Vision-Language) . . . 48
8.9 DeepSeek Design Philosophy: A Summary . . . 49

9 Claude: Anthropic's Safety-First Frontier Models 51
9.1 Constitutional AI (Bai et al., December 2022) . . . 51
9.1.1 The Paper . . . 51
9.1.2 The Constitutional AI Method . . . 51
9.2 Claude 1 (March 2023) . . . 52
9.2.1 The Model . . . 52
9.3 Claude 2 (July 2023) . . . 52
9.3.1 Improvements . . . 52
9.4 Claude 3 (March 2024) . . . 52
9.4.1 The Family . . . 52
9.4.2 Architectural Clues (From Anthropic's Research) . . . 52
9.4.3 Results . . . 52
9.5 Claude 3.5 (June 2024) . . . 53
9.5.1 Claude 3.5 Sonnet . . . 53
9.5.2 Claude 3.5 Haiku (October 2024) . . . 53
9.5.3 Computer Use (October 2024) . . . 53
9.5.4 Claude 3.5 Opus (Cancelled?) . . . 53
9.6 Anthropic's Key Research Papers . . . 53
9.6.1 Training a Helpful and Harmless Assistant (Bai et al., 2022) . . . 53
9.6.2 Towards Monosemanticity: Decomposing Language Models With Dictionary Learning (Dec 2023) . . . 53
9.6.3 Scaling Monosemanticity (May 2024) . . . 54
9.6.4 The Claude Model Card (March 2024/2025) . . . 54

9.7 Anthropic's Philosophy vs. OpenAI . . . 54

10 MoE Architectures Beyond DeepSeek **55**
10.1 DBRX (Databricks, March 2024) . . . 55
10.1.1 The Paper . . . 55
10.1.2 Architecture . . . 55
10.1.3 Key Innovation: No Token Dropping . . . 55
10.1.4 Results . . . 55
10.2 Jamba (AI21 Labs, March 2024) . . . 56
10.2.1 The Paper . . . 56
10.2.2 Architecture: Mamba + Transformer + MoE . . . 56
10.2.3 Key Innovation: Hybrid Architecture . . . 56
10.3 Qwen2-MoE (Alibaba, June 2024) . . . 56
10.4 Grok-1 (xAI, March 2024) . . . 56
10.5 Switch Transformer (Fedus et al., 2022) . . . 57
10.6 GShard (Lepikhin et al., 2021) . . . 57
10.7 MoE Design Space . . . 57

11 State Space Models and Linear-Complexity Alternatives **59**
11.1 Mamba (Gu & Dao, December 2023) . . . 59
11.1.1 The Paper . . . 59
11.1.2 The Core Problem: Attention is Quadratic . . . 59
11.1.3 Mamba's Innovation: Selective SSM . . . 59
11.1.4 Hardware-Aware Algorithm . . . 59
11.1.5 Architecture Details . . . 60
11.1.6 Mamba-2 (Dao & Gu, May 2024) . . . 60
11.2 RWKV: Receptance Weighted Key Value (Peng et al., 2023) . . . 60
11.2.1 The Paper . . . 60
11.2.2 Architecture . . . 60
11.2.3 Training = Parallel, Inference = RNN . . . 61
11.2.4 RWKV Evolution . . . 61
11.3 Other Linear-Complexity Models . . . 61
11.3.1 Griffin (DeepMind, March 2024) . . . 61
11.3.2 Hyena (Poli et al., 2023) . . . 61
11.3.3 H3 (Fu et al., 2023) . . . 61
11.3.4 xLSTM (Beck et al., 2024) . . . 61
11.3.5 Falcon-Mamba 7B (TII, 2024) . . . 62
11.4 The Alternative Architecture Landscape . . . 62

12 Code-Specialized LLMs **63**
12.1 CodeLlama (Meta, August 2023) . . . 63
12.1.1 The Paper . . . 63
12.1.2 Approach . . . 63
12.1.3 Specialized Training . . . 63
12.1.4 Results . . . 64
12.2 StarCoder (BigCode, May 2023–Feb 2024) . . . 64
12.2.1 StarCoder 1 (BigCode, May 2023) . . . 64
12.2.2 StarCoder 2 (BigCode + ServiceNow, Feb 2024) . . . 64
12.3 DeepSeek-Coder Series . . . 64

12.3.1 DeepSeek-Coder V1 (November 2023) . . . 64
12.3.2 DeepSeek-Coder V2 (June 2024) . . . 65
12.4 Other Notable Code Models . . . 65
12.4.1 WizardCoder (Luo et al., 2023) . . . 65
12.4.2 CodeQwen 1.5 (Alibaba, 2024) . . . 65
12.4.3 Magicoder (Wei et al., 2024) . . . 65
12.4.4 CodeGemma (Google, 2024) . . . 65
12.5 Code Model Design Patterns . . . 65

13 Community and Open-Source LLM Milestones 67
13.1 Falcon (TII, UAE, 2023) . . . 67
13.1.1 Falcon-40B and 180B (June–November 2023) . . . 67
13.2 MPT (MosaicML, May 2023) . . . 67
13.2.1 The Paper . . . 67
13.2.2 Architecture . . . 67
13.2.3 Context Extension . . . 68
13.3 Yi (01.AI, November 2023) . . . 68
13.3.1 The Paper . . . 68
13.3.2 Architecture . . . 68
13.3.3 Results . . . 68
13.4 OLMo (AI2, February 2024) . . . 68
13.4.1 The Paper . . . 68
13.4.2 Full Transparency . . . 69
13.5 Command R / Command R+ (Cohere, 2024) . . . 69
13.5.1 Architecture . . . 69
13.6 BLOOM (BigScience, July 2022) . . . 69
13.6.1 The Paper . . . 69
13.6.2 Architecture . . . 69
13.7 OPT (Meta, May 2022) . . . 70
13.7.1 The Paper . . . 70
13.7.2 Architecture . . . 70
13.8 Amazon: Titan, Olympus, Nova . . . 70
13.9 Other Notable Models . . . 70

14 Instruction-Tuned and Preference-Aligned Models 71
14.1 Vicuna (LMSYS, March 2023) . . . 71
14.1.1 The Paper . . . 71
14.1.2 Approach . . . 71
14.1.3 Significance . . . 71
14.2 WizardLM (Microsoft, April 2023) . . . 71
14.2.1 The Paper . . . 71
14.2.2 Evol-Instruct Method . . . 72
14.2.3 WizardCoder / WizardMath . . . 72
14.3 Orca (Microsoft, June 2023) . . . 72
14.3.1 The Paper . . . 72
14.3.2 Progressive Learning . . . 72
14.3.3 Results . . . 72
14.4 Zephyr (HuggingFace, October 2023) . . . 73

14.4.1 The Paper . . . 73
14.4.2 Three-Stage Alignment . . . 73
14.4.3 Results . . . 73
14.5 NeuralChat (Intel, 2023–2024) . . . 73
14.6 UltraChat / UltraFeedback (2023–2024) . . . 73
14.6.1 UltraChat (Ding et al., 2023) . . . 73
14.6.2 UltraFeedback (Cui et al., 2023) . . . 73
14.7 Airoboros (2023) . . . 74
14.8 Alignment Model Comparison . . . 74

15 Training Paradigms and Scaling Innovations 75
15.1 The Scaling Laws (Kaplan et al., 2020) . . . 75
15.1.1 The Paper . . . 75
15.1.2 Key Findings . . . 75
15.2 Chinchilla: Compute-Optimal Training (Hoffmann et al., 2022) . . . 76
15.2.1 Revision of Kaplan's Laws . . . 76
15.2.2 Why Kaplan's Laws Were Wrong . . . 76
15.3 Maximal Update Parameterization (μP, Yang et al., 2022) . . . 76
15.3.1 The Paper . . . 76
15.3.2 The Problem . . . 76
15.3.3 The Solution: μP . . . 76
15.4 Data Curation and Filtering Innovations . . . 77
15.4.1 Deduplication . . . 77
15.4.2 Quality Filtering . . . 77
15.4.3 Data Mixing and Curriculum . . . 77
15.5 Training Infrastructure Innovations . . . 77
15.5.1 3D Parallelism (Megatron-LM, DeepSpeed) . . . 77
15.5.2 FSDP (Fully Sharded Data Parallel, PyTorch) . . . 77
15.5.3 Mixed Precision Training . . . 78
15.5.4 Activation Checkpointing / Gradient Checkpointing . . . 78
15.5.5 Sequence Packing . . . 78
15.6 Beyond Next-Token Prediction . . . 78
15.6.1 Multi-Token Prediction (Gloeckle et al., 2024; DeepSeek-V3) . . . 78
15.6.2 UL2 (Tay et al., 2022)—Mixture of Denoisers . . . 78
15.6.3 FIM (Fill-in-the-Middle) . . . 78
15.6.4 Knowledge Distillation in Pre-Training . . . 78
15.7 Training Efficiency Summary . . . 79

Chapter 1

The GPT Family: Autoregressive Language Models

The GPT series defined the modern LLM paradigm: large-scale autoregressive transformers pre-trained on massive text corpora, then fine-tuned or prompted for downstream tasks. This chapter traces the full lineage.

1.1 GPT-1: "Improving Language Understanding by Generative Pre-Training" (Radford et al., 2018)

1.1.1 The Paper and Problem

Paper: Radford, A., Narasimhan, K., Salimans, T., & Sutskever, I. (2018). *Improving Language Understanding by Generative Pre-Training.* OpenAI technical report.

Problem Solved: Before GPT-1, NLP relied on task-specific architectures and hand-labeled data. The key insight: a generative language model pre-trained on unlabeled text could serve as a universal feature extractor, requiring only minimal task-specific adaptation.

1.1.2 Architecture

- **Base:** 12-layer transformer **decoder-only** (no encoder). This was a critical design choice—the decoder-only architecture enables natural autoregressive generation.
- **Parameters:** 117M. Hidden size $d = 768$, 12 attention heads, context length 512.
- **Activation:** GELU (Gaussian Error Linear Unit).
- **Position encoding:** Learned absolute position embeddings.
- **Byte-Pair Encoding (BPE):** 40,000 merge operations vocabulary.
- **Pre-norm:** Layer norm applied *before* each sub-layer.

1.1.3 Training

- **Pre-training:** Standard language modeling on BooksCorpus (7,000 unpublished books, ∼5 GB).
- **Objective:** Maximize $\sum_i \log P(u_i \mid u_{i-k}, \ldots, u_{i-1}; \Theta)$ with $k = 512$.
- **Optimizer:** Adam, max LR 2.5×10^{-4}, 100 epochs.

1.1.4 Fine-tuning Approach (The "GPT Paradigm")

The core contribution was the **task-agnostic input transformation**:

- All tasks (classification, entailment, similarity, QA) are reformulated as text sequences with special delimiters: `[Start]`, `[Delim]`, `[Extract]`.
- A single linear classifier head is added on top of the final transformer block's output for the last token.
- **Auxiliary LM objective** during fine-tuning: $\mathcal{L} = \mathcal{L}_{\text{task}} + \lambda \cdot \mathcal{L}_{\text{LM}}$ improves generalization and convergence speed.

1.1.5 Novelty and Impact

- **Decoder-only pre-training:** Established the paradigm that a unidirectional transformer could serve as a universal NLP backbone.
- **Task-agnostic fine-tuning:** Single architecture, minimal task-specific parameters. No need for task-specific architectures (LSTMs for NER, tree-LSTMs for entailment, etc.).
- **Results:** 72.8 on RACE (previous SOTA: 59.0), 88.1 F1 on CoLA. 5–8% absolute improvement across 9 of 12 tasks over task-specific models.

1.2 GPT-2: "Language Models are Unsupervised Multitask Learners" (Radford et al., 2019)

1.2.1 The Paper and Problem

Paper: Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., & Sutskever, I. (2019). *Language Models are Unsupervised Multitask Learners.*

Problem Solved: Can a language model, without any task-specific fine-tuning, perform diverse NLP tasks? The answer was yes—**zero-shot transfer**.

1.2.2 Architectural Evolution

- **Scaling:** 1.5B parameters (largest variant). Hidden size 1600, 48 layers, 25 heads.
- **Normalization:** Layer normalization moved to the input of each sub-block (pre-activation), plus an additional layer norm after the final self-attention.
- **Weight tying:** Input token embeddings tied to output weight matrix.
- **Modified initialization:** Residual layer weights scaled by $1/\sqrt{N}$ where N is the number of residual layers.
- **Vocabulary:** 50,257 tokens. Context length extended to 1024.
- **Batch size:** 512, trained on WebText (8M documents, 40 GB from outbound Reddit links).

1.2.3 Zero-Shot Transfer Paradigm

GPT-2's defining innovation was the **zero-shot** approach:

- No fine-tuning on any downstream task.
- Tasks are specified purely through natural language prompts: e.g., "Translate to French: `text` => `translation`".

- The model must infer the task from the prompting pattern alone.
- Results: 63.1 F1 on CoQA (matching supervised baselines); 55.5 BLEU on WMT-14 Fr-En (without seeing any parallel text).

1.2.4 Why GPT-2 was Initially Withheld

OpenAI cited concerns about misuse for generating deceptive content. The staged release (124M $\rightarrow$ 355M $\rightarrow$ 774M $\rightarrow$ 1.5B) was a landmark moment in AI safety discourse.

1.3 GPT-3: "Language Models are Few-Shot Learners" (Brown et al., 2020)

1.3.1 The Paper

Paper: Brown, T.B., Mann, B., Ryder, N., *et al.* (2020). *Language Models are Few-Shot Learners.* NeurIPS 2020.

Core Question: Can scaling alone unlock few-shot learning, without gradient updates?

1.3.2 Architecture Scaling

- **Scale jump:** 175B parameters (vs GPT-2's 1.5B—a 116$\times$ increase). 96 layers, $d = 12288$, 96 heads.
- **Architecture:** Same as GPT-2 but with **alternating dense and locally banded sparse attention patterns** in different layers (similar to Sparse Transformer, Child et al., 2019). This is the GPT-3 unique architectural twist—not all layers use full dense attention.
- **Training data:** 570 GB of filtered Common Crawl, WebText2, Books1/2, Wikipedia. Total $\sim$300B tokens.
- **Training cost:** Estimated $\sim$4.6M USD in compute (V100 GPU-years).

1.3.3 The "In-Context Learning" Discovery

GPT-3's key contribution was demonstrating and naming **in-context learning**:

- **Zero-shot:** Task description only. "Translate English to French: cheese =>".
- **One-shot:** One example in the prompt. "Translate English to French: sea otter => loutre de mer. cheese =>".
- **Few-shot:** 10–100 examples. Model "learns" the pattern without weight updates.

GPT-3 showed a smooth power-law improvement in few-shot performance with model scale—the "scaling curve" that became a central research theme.

1.3.4 Key Results

- 69.3% on SuperGLUE few-shot (prior SOTA: 71.8% with fine-tuning).
- 81.5 F1 on TriviaQA few-shot (approaching SOTA fine-tuned).
- Generated coherent news articles that human evaluators found difficult to distinguish from human-written ones (52% human detection rate vs 50% random).

1.3.5 Novelty Summary

- **In-context learning** as an emergent phenomenon of scale.
- Demonstrated **meta-learning**: the model learns to learn from examples within its context window.
- Sparse attention patterns for computational efficiency at scale.
- 8 model sizes (125M to 175B) providing the best scaling-law data point before Chinchilla.

1.4 GPT-3.5, InstructGPT, and ChatGPT

1.4.1 InstructGPT (Ouyang et al., 2022)

Paper: Ouyang, L., *et al.* (2022). *Training Language Models to Follow Instructions with Human Feedback.* NeurIPS 2022.

Problem: GPT-3 often generated untruthful, toxic, or unhelpful outputs. It was trained to predict internet text, not to be helpful.

Solution — RLHF for Alignment:

1. **Step 1—SFT:** Collect human-written demonstrations of desired behavior (∼13K prompt-response pairs). Supervised fine-tune GPT-3.
2. **Step 2—RM training:** Collect comparison data (for each prompt, rank 4–9 model outputs). Train a reward model (also 6B GPT-3) to predict human preferences.
3. **Step 3—PPO:** Optimize the SFT policy against the reward model using Proximal Policy Optimization, with a KL penalty preventing divergence from the SFT policy.

Result: 1.3B InstructGPT preferred over 175B GPT-3 by human evaluators. Demonstrated that alignment is more important than raw scale for user satisfaction.

1.4.2 ChatGPT (November 2022)

Built on GPT-3.5 (an improved GPT-3 variant) with RLHF. The model that brought LLMs to the mainstream. Used a conversational interface with multi-turn dialogue support. Key technical elements:

- Conversation history included in prompt.
- RLHF optimized for helpfulness, harmlessness, and honesty (HHH).
- System-level safety filters and content moderation.
- Iterative deployment with user feedback collection.

1.5 GPT-4: "GPT-4 Technical Report" (OpenAI, 2023)

1.5.1 The Paper

Paper: OpenAI (2023). *GPT-4 Technical Report.* Notably, this paper deliberately omitted architecture details and model size, citing competitive and safety concerns.

1.5.2 What We Know

- **Multimodal:** Accepts images and text as input; outputs text. Vision encoded via a separate vision transformer, then projected into the language space.

- **Mixture of Experts (likely):** Based on inference patterns and community analysis, GPT-4 is believed to use a MoE architecture with 8 experts and ∼1.8T total parameters (unconfirmed).
- **Training:** Pre-trained on a mixture of publicly available and licensed data. Then RLHF + rule-based reward models.
- **Context length:** 8K tokens initial; 32K variant later released.
- **Predictable scaling:** OpenAI developed infrastructure to predict final model performance from small-scale training runs using scaling laws (improving on Kaplan et al., 2020).

1.5.3 Predictable Scaling (Key Innovation)

The report details a novel capability: predicting GPT-4's final loss from models trained with 1/1000× compute. This was used to optimize hyperparameters and architecture decisions before committing to the full training run. The method fits a scaling law:

$$L(C) = L_0 + \left(\frac{C_0}{C}\right)^{\alpha} \tag{1.1}$$

where C is compute budget and parameters are fitted from small-scale runs.

1.5.4 Key Results

- 90th percentile on Uniform Bar Exam (vs 10th for GPT-3.5).
- 86.4% on MMLU (5-shot).
- Passes AP exams across multiple subjects in the 4–5 range.
- Significantly reduced hallucination rate vs GPT-3.5.

1.6 GPT-4o: Omni-Modal Model (OpenAI, May 2024)

1.6.1 The Model

Announcement: May 2024. "o" stands for "omni"—truly multimodal input and output.

1.6.2 Architectural Innovations

- **Natively multimodal:** Processes and generates text, images, and audio in a single unified model (not separate encoders pipelined together).
- **Real-time voice:** Audio-to-audio latency of 232 ms (avg), 320 ms max. Previous voice mode used cascaded ASR → LLM → TTS with 2.8s (GPT-3.5) or 5.4s (GPT-4) latency.
- **Token-level multimodality:** The model operates on a unified token space—text tokens, vision tokens, and audio tokens are interleaved in the same sequence.
- **Emotion and tone understanding:** Can perceive speaker emotion, background noises, and tone in audio. Output speech with adjustable tone, speed, and emotional expressiveness.

1.6.3 The Tokenizer Innovation

GPT-4o introduced a new tokenizer capable of efficiently encoding non-text modalities. For images: variable-size tiling (not fixed 224×224 or 336×336 patches). For audio: the model directly encodes raw audio into discrete tokens in the shared vocabulary space.

1.6.4 o1 and o3: Reasoning Models

OpenAI's reasoning-focused model family (September–December 2024):

- **Chain-of-thought at training time:** Models are trained to generate long internal reasoning chains before producing final answers.
- **Test-time compute scaling:** More inference-time computation (longer reasoning) produces better answers—a new scaling axis beyond model size.
- **RL-based reasoning:** Trained with reinforcement learning to produce correct reasoning steps, not just correct final answers.
- **o1:** First reasoning model. Strong at math, coding, and science (comparable to PhD students on GPQA).
- **o3:** Dramatically improved. 25.2% on FrontierMath, 87.5% on ARC-AGI (bested prior SOTA of 55.5%). Introduces "program search" over reasoning chains.

1.7 Summary: The GPT Architectural Lineage

Table 1.1: GPT architectural evolution

Model	Year	Params	Key Innovation	Paper
GPT-1	2018	117M	Decoder-only pre-training + task-agnostic FT	Radford et al., 2018
GPT-2	2019	1.5B	Zero-shot transfer, scaled decoder-only	Radford et al., 2019
GPT-3	2020	175B	In-context learning, sparse attention	Brown et al., 2020
InstructGPT	2022	1.3B/6B	RLHF alignment (SFT + RM + PPO)	Ouyang et al., 2022
GPT-4	2023	Unk.	Multimodal input, predictable scaling	OpenAI, 2023
GPT-4o	2024	Unk.	Natively omni-modal, real-time audio	OpenAI, 2024
o1/o3	2024	Unk.	Chain-of-thought RL, test-time compute	OpenAI, 2024

Chapter 2

The BERT Family: Bidirectional Encoder Models

While GPT popularized decoder-only models, the BERT family established the encoder-only paradigm for NLU tasks. This chapter traces BERT through its most influential variants.

2.1 BERT: Bidirectional Encoder Representations from Transformers (Devlin et al., 2019)

2.1.1 The Paper

Paper: Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* NAACL 2019.

Problem: GPT's unidirectional (left-to-right) pre-training limited its NLU capabilities. Language understanding requires integrating context from both directions. Existing bidirectional models (ELMo) used shallow concatenation of left-to-right and right-to-left LSTMs, not deep bidirectional attention.

2.1.2 Architecture

- **Base:** Multi-layer bidirectional **Transformer encoder** (not decoder). This is the fundamental difference from GPT.
- **BERT-Base:** $L = 12$, $H = 768$, $A = 12$, 110M parameters.
- **BERT-Large:** $L = 24$, $H = 1024$, $A = 16$, 340M parameters.
- **Position embeddings:** Learned absolute position embeddings.
- **Activation:** GELU.
- **Vocabulary:** 30,000 WordPiece tokens. Context length 512.
- **Feed-forward:** 4× hidden size (standard for the era).

2.1.3 Pre-Training Objectives (The Dual Objective Innovation)

BERT introduced two complementary pre-training objectives:

1. Masked Language Modeling (MLM): Randomly mask 15% of input tokens. Of those 15%:

- 80% replaced with `[MASK]`.
- 10% replaced with a random token.
- 10% left unchanged.

The model predicts the original token at masked positions. This enables deep bidirectional context—the model must integrate left AND right context to predict the masked word.

2. Next Sentence Prediction (NSP): Given two sentences A and B, predict whether B is the actual next sentence following A (50% of training data) or a random sentence (50%). This objective was designed to help with QA and NLI tasks that require sentence-level reasoning. *Note:* Later work (RoBERTa) showed NSP is not actually necessary.

2.1.4 Fine-Tuning

Unlike GPT's generative pre-training, BERT is designed for **discriminative** tasks:

- **Single sentence classification:** `[CLS]` token's final hidden state fed to a classifier.
- **Sentence pair:** `[CLS]` A `[SEP]` B `[SEP]`.
- **Token-level (NER, QA):** Each token's final hidden state fed to a token-level classifier.

2.1.5 Results and Impact

- 7.7% absolute improvement on GLUE over prior SOTA.
- 93.2 F1 on SQuAD v1.1, 83.1 on SQuAD v2.0.
- BERT established the **pre-train then fine-tune** paradigm as the NLP standard.
- 50,000+ citations, widely used as the basis for domain-specific LMs.

2.2 RoBERTa: A Robustly Optimized BERT Approach (Liu et al., 2019)

2.2.1 The Paper

Paper: Liu, Y., Ott, M., Goyal, N., *et al.* (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach.*

Problem: BERT was significantly **undertrained**. RoBERTa demonstrated that BERT's architecture could achieve much better results simply through better training methodology.

2.2.2 Changes from BERT

1. **Longer training:** Trained on 160 GB of text (vs BERT's 16 GB) for 500K steps (vs BERT's 1M). Much larger batch sizes (8K sequences).
2. **Dynamic masking:** Generate new masking patterns each epoch (BERT used static masking—same pattern every epoch).
3. **Removed NSP:** Train with FULL-SENTENCES format (multiple sentences packed to fill 512 tokens). No next-sentence prediction objective. Demonstrated NSP is not necessary.

4. **Larger byte-level BPE vocabulary:** 50K subword units (vs BERT's 30K WordPiece).
5. **Training data:** BOOKCORPUS + CC-NEWS + OPENWEBTEXT + STORIES.
6. **Longer sequences:** Full-length sequences only (no short sequences padded to 512).

2.2.3 Results

- 88.5 on GLUE (vs 83.2 for BERT-Large).
- 90.2 on SQuAD (vs 88.5 for BERT-Large).
- **Key lesson:** Training methodology matters as much as architecture. Hyperparameter tuning and data scale can dramatically improve the same architecture.

2.3 ALBERT: A Lite BERT (Lan et al., 2019)

2.3.1 The Paper

Paper: Lan, Z., Chen, M., Goodman, S., *et al.* (2019). *ALBERT: A Lite BERT for Self-supervised Learning of Language Representations.* ICLR 2020.

Problem: BERT models hit GPU memory limits (cannot scale past BERT-Large on single GPU). How to scale model capacity without scaling memory?

2.3.2 Key Innovations

1. Factorized Embedding Parameterization: BERT's embedding matrix is $V \times H$ (vocab $\times$ hidden). ALBERT decomposes this into two smaller matrices:

$$\mathbf{E} = \mathbf{E}_{\text{proj}} \cdot \mathbf{E}_{\text{vocab}}, \quad \mathbf{E}_{\text{vocab}} \in \mathbb{R}^{V \times E}, \quad \mathbf{E}_{\text{proj}} \in \mathbb{R}^{E \times H} \tag{2.1}$$

where $E \ll H$ (e.g., $E = 128$, $H = 768$). This reduces embedding parameters from $O(VH)$ to $O(VE + EH)$, saving $\sim$20M parameters for BERT-Base.

2. Cross-Layer Parameter Sharing: All transformer layers share the same parameters (feed-forward and attention weights). The forward pass becomes $\mathbf{H}^{(\ell+1)} = f_\theta(\mathbf{H}^{(\ell)})$ for all ℓ. This reduces parameters by factor L, but doesn't reduce compute.

3. Sentence-Order Prediction (SOP): Replaces NSP with a more challenging objective: predict whether two consecutive segments are in the correct order or swapped. This is strictly harder than NSP (which only requires topic coherence).

2.3.3 Results

- ALBERT-xxlarge (235M params with sharing, would be $\sim$1.2B without sharing).
- Matched BERT-Large on GLUE with 18$\times$ fewer parameters (when controlling for compute, not parameter count).
- **Limitation:** Parameter sharing reduces memory but not compute. Same inference latency as a dense model.

2.4 DistilBERT: Distilled BERT (Sanh et al., 2019)

2.4.1 The Paper

Paper: Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). *DistilBERT, a Distilled Version of BERT: Smaller, Faster, Cheaper and Lighter.* NeurIPS 2019 Workshop.

Problem: BERT-Base (110M params, 1.6 GB) was too large for edge deployment.

2.4.2 Architecture

- 6 layers (half of BERT-Base), same hidden size. Student initialized from every other layer of BERT-Base.
- **Triple loss during pre-training distillation:**

$$\mathcal{L} = \mathcal{L}_{\text{MLM}} + \mathcal{L}_{\text{KD}} + \mathcal{L}_{\text{COS}} \tag{2.2}$$

  where $\mathcal{L}_{\text{KD}}$ is the standard KL-divergence knowledge distillation and $\mathcal{L}_{\text{COS}}$ minimizes cosine distance between teacher and student hidden states.
- Removes token-type embeddings and pooler.
- 66M parameters, 97% of BERT-Base performance on GLUE.

2.5 DeBERTa: Decoding-enhanced BERT with Disentangled Attention (He et al., 2021)

2.5.1 The Paper

Paper: He, P., Liu, X., Gao, J., & Chen, W. (2021). *DeBERTa: Decoding-enhanced BERT with Disentangled Attention.* ICLR 2021.

2.5.2 Key Innovation: Disentangled Attention

Standard attention computes $\mathbf{QK}^\top$ where $\mathbf{q}_i = \mathbf{x}_i\mathbf{W}_Q$, $\mathbf{k}_j = \mathbf{x}_j\mathbf{W}_K$. DeBERTa decomposes each word into **content** and **position** vectors:

$$\mathbf{A}_{ij} = \mathbf{q}_{c,i}^\top \mathbf{k}_{c,j} + \mathbf{q}_{c,i}^\top \mathbf{k}_{p,(j-i)} + \mathbf{q}_{p,(j-i)}^\top \mathbf{k}_{c,j} \tag{2.3}$$

where $\mathbf{q}_{p,\delta}$ and $\mathbf{k}_{p,\delta}$ are learned position embeddings for relative distance δ. This has 3 terms: content-content, content-position, and position-content (no position-position term, which was found to be less useful).

2.5.3 Enhanced Mask Decoder (EMD)

A two-layer attention decoder added *on top of* the encoder, trained during pre-training. At each layer:

$$\mathbf{h}_i = \text{LayerNorm}(\mathbf{h}_i + \text{DeBERTa-Attention}(\mathbf{h}_i)) \tag{2.4}$$

The decoder uses absolute position embeddings (unlike the encoder's relative) and is trained via the MLM objective. This "decoding-enhanced" aspect gives DeBERTa its name.

2.5.4 DeBERTaV3 (He et al., 2023)

Replaces MLM with **RTFD (Replaced Token Detection + ELECTRA-style)**:

- A generator (like ELECTRA's) replaces ∼15% of tokens with plausible alternatives using a small MLM head.
- The main DeBERTa model acts as a **discriminator**, predicting which tokens were replaced.
- This is a more efficient pre-training signal than MLM since every token provides a learning signal (not just 15%).
- **Gradient-disentangled embedding sharing (GDES):** Shares embeddings between generator and discriminator but stops gradient flow from discriminator to generator embeddings, preventing interference.

DeBERTaV3 achieved 91.6 on SuperGLUE, surpassing human baselines on several tasks.

2.5.5 Summary: BERT Architectural Contributions

Table 2.1: Key BERT variants and their contributions

Model	Year	Key Contribution
BERT	2019	Bidirectional pre-training with MLM+NSP; fine-tuning paradigm
RoBERTa	2019	Dynamic masking, remove NSP, better training recipe
ALBERT	2019	Factorized embeddings, cross-layer parameter sharing
DistilBERT	2019	Knowledge distillation for 40% smaller models
DeBERTa	2021	Disentangled attention (content vs position); enhanced mask decoder
DeBERTaV3	2023	ELECTRA-style RTD pre-training, gradient-disentangled embedding sharing

Chapter 3

T5, PaLM, and Google's Training Breakthroughs

Google contributed foundational encoder-decoder architectures, extreme-scale dense models, and the Chinchilla scaling laws.

3.1 T5: Text-to-Text Transfer Transformer (Raffel et al., 2020)

3.1.1 The Paper

Paper: Raffel, C., Shazeer, N., Roberts, A., *et al.* (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.* JMLR 2020.

Core Philosophy: Every NLP task is a text-to-text problem. Input: text. Output: text. No task-specific heads, no architecture modifications.

3.1.2 Architecture

- **Encoder-decoder transformer** (standard Transformer architecture from Vaswani et al., 2017).
- Sizes: Small (60M) to 11B.
- **Relative position embeddings:** Instead of absolute or sinusoidal, T5 uses simplified relative position biases—learned scalar biases added to attention logits based on relative distance between query and key:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{B}\right)\mathbf{V} \tag{3.1}$$

  where $\mathbf{B}_{ij} = b_{\text{clip}(j-i,-k,k)}$ is a learned scalar for each relative distance bucket.

3.1.3 The "Colossal Clean Crawled Corpus" (C4)

750 GB of cleaned English web text. The data pipeline became a reference standard.

3.1.4 Span Corruption Pre-Training

Unique pre-training objective: mask random spans of text (not individual tokens) and train the model to reconstruct them:

```
Input:  "Thank you for <X> me to your party <Y> week."
Target: "<X> inviting <Y> last <Z>"
(where <X>, <Y>, <Z> are sentinel tokens)
```

This span-corruption approach proved more effective than BERT-style token masking for generative tasks.

3.1.5 Key Findings (The Ablation Study)

T5's paper is famous for its systematic ablations:

- Encoder-decoder slightly outperforms decoder-only at equal compute for NLU $\rightarrow$ NLG tasks.
- Pre-training on 1T tokens is better than on 100B tokens (more data helps, no saturation).
- Multi-task pre-training (mixing many tasks) underperforms unsupervised pre-training + fine-tuning.

3.2 PaLM: Pathways Language Model (Chowdhery et al., 2022)

3.2.1 The Paper

Paper: Chowdhery, A., Narang, S., Devlin, J., *et al.* (2022). *PaLM: Scaling Language Modeling with Pathways.* arXiv:2204.02311.

3.2.2 Architecture (PaLM's Innovations)

1. Parallel Attention and FFN: Standard transformers compute attention and FFN sequentially. PaLM computes them in parallel, with separate RMSNorm for each:

$$\mathbf{H}^{(\ell+1)} = \mathbf{H}^{(\ell)} + \text{Attn}(\text{RMSNorm}_{\text{attn}}(\mathbf{H}^{(\ell)})) + \text{FFN}(\text{RMSNorm}_{\text{ffn}}(\mathbf{H}^{(\ell)})) \tag{3.2}$$

This removes the sequential dependency, increasing training throughput by $\sim$15% at scale. The parallel formulation was found to improve training stability.

2. Multi-Query Attention (MQA): All heads share a single $\mathbf{K}, \mathbf{V}$ projection. This was the first major LLM to adopt MQA, dramatically reducing KV-cache memory:

$$\text{head}_i = \text{Attention}(\mathbf{Q}_i, \mathbf{K}_{\text{shared}}, \mathbf{V}_{\text{shared}}) \tag{3.3}$$

3. SwiGLU Activation: Adopted the SwiGLU FFN from Shazeer (2020).

4. RoPE: Used Rotary Positional Embeddings.

5. Shared Input-Output Embeddings: Reduced parameter count.

6. Vocabulary: 256K SentencePiece tokens (trained on lossless vocabulary optimization).

3.2.3 Training System: Pathways

PaLM was trained using Google's **Pathways** system, which enables:

- Training across 6144 TPU v4 chips.
- Two pod slices connected via data center network (DCN), not all-to-all TPU interconnect.
- Achieved 57.8% model FLOPs utilization (MFU)—highest at the time for this scale.

3.2.4 Results

- 540B parameters. Trained on 780B tokens.
- First model to demonstrate **discontinuous improvements** (emergent abilities)—certain capabilities appear suddenly at scale, not gradually.
- 62.4% on BIG-Bench (5-shot), exceeding average human performance on several tasks.
- Training was "fire and forget"—no loss spikes, no restarts. A significant engineering achievement.

3.2.5 Variants

- **Med-PaLM 2:** Fine-tuned on medical data. First LLM to achieve "expert" level on USMLE-style medical exam questions.
- **PaLM 2:** Improved version with better multilingual performance and reasoning.

3.3 Chinchilla: Training Compute-Optimal LLMs (Hoffmann et al., 2022)

3.3.1 The Paper

Paper: Hoffmann, J., Borgeaud, S., Mensch, A., *et al.* (2022). *Training Compute-Optimal Large Language Models.* NeurIPS 2022.

Problem: Prior scaling laws (Kaplan et al., 2020) suggested that model size should grow faster than training data. Chinchilla demonstrated this was wrong.

3.3.2 The Chinchilla Scaling Law

Given a fixed compute budget C FLOPs, the optimal allocation between parameters N and training tokens D is:

$$N_{\text{opt}}(C) \propto C^{0.49}, \quad D_{\text{opt}}(C) \propto C^{0.51} \tag{3.4}$$

Surprisingly, parameters and tokens should grow at approximately equal rates. For $C = 6 \times 10^{23}$ (Chinchilla's budget):

- Kaplan's approach: 280B parameters, 293B tokens $\rightarrow$ suboptimal.
- Chinchilla's approach: 70B parameters, 1.4T tokens. 20 tokens per parameter.

3.3.3 Chinchilla Model

Trained according to the scaling law:

- 70B parameters. Trained on 1.4T tokens (vs Gopher's 280B/300B).
- **Outperforms Gopher (280B)** on most tasks despite being 4× smaller.
- **Outperforms GPT-3 (175B)** with less than half the parameters.

3.3.4 The Key Insight

Most LLMs are significantly undertrained. The Chinchilla-optimal ratio is ∼20 tokens per parameter. Real-world models:

- GPT-3 175B: ∼300B tokens (1.7 token/param—massively undertrained).
- LLaMA 65B: ∼1.4T tokens (21.5 tok/param—near Chinchilla-optimal).
- DeepSeek-V2: 8.1T tokens for 236B (not MoE) → far beyond Chinchilla for dense.

3.3.5 Later Revision: Chinchilla is Not a Ceiling

Subsequent work (Sardana et al., 2023; DeepSeek, 2024) showed that training BEYOND Chinchilla-optimal still improves performance, just with diminishing returns. The Chinchilla point is a trade-off minimum (best performance per compute), not a saturation ceiling. DeepSeek-V3 trains ∼22T effective tokens, well beyond any Chinchilla estimate.

3.4 Gemini: Google's Multimodal Frontier (2023–2024)

3.4.1 Gemini 1.0 (December 2023)

Paper: Gemini Team, Google (2023). *Gemini: A Family of Highly Capable Multimodal Models.*

Architecture: Natively multimodal—processes text, images, audio, and video in a unified model. Sizes: Ultra (∼1.5T+?), Pro, Nano (1.8B and 3.25B for on-device).

Key innovations:

- Early fusion of modalities (not separate encoders). Interleaved multimodal tokens.
- **Retrieval-augmented training:** The model has access to a retrieval system during training, not just inference.
- **Co-training on TPUv5p:** Google's latest TPU generation. Designed for multimodal tensor operations.

3.4.2 Gemini 1.5 (February 2024)

Massive context window expansion to 1M tokens (10M in research version). Achieved via:

- **Ring Attention + Sparse Attention:** Distributed long-context processing.
- **Mixture of Depths (?):** Possible attention sparsity across layers.
- The model can process entire codebases, multi-hour videos, or complete book series in context.

3.4.3 Gemini 2.0 (December 2024–2025)

Agentic AI era. Native tool use. Supports Google Search, code execution, and API calls as native capabilities. Flash variant optimized for speed; Pro variant for depth.

Chapter 4

The LLaMA Family: Democratizing Open-Weight LLMs

Meta's LLaMA series catalyzed the open-weight LLM revolution. Each iteration introduced architectural optimizations that became industry standards.

4.1 LLaMA 1: Open and Efficient Foundation Language Models (Touvron et al., 2023)

4.1.1 The Paper

Paper: Touvron, H., Lavril, T., Izacard, G., *et al.* (2023). *LLaMA: Open and Efficient Foundation Language Models.*

Problem: Prior open models (OPT, BLOOM) underperformed closed models. LLaMA aimed to match GPT-3 quality using only publicly available data and more efficient architecture.

4.1.2 Architectural Innovations

LLaMA introduced several architecture improvements that became standard:

1. Pre-normalization (RMSNorm): Replaced LayerNorm with RMSNorm (Zhang & Sennrich, 2019) for training stability and efficiency. Applied *before* each sub-layer (not after):

$$\text{RMSNorm}(\mathbf{x}) = \mathbf{x} \cdot \frac{\boldsymbol{\gamma}}{\sqrt{\frac{1}{d}\sum x_i^2 + \epsilon}} \tag{4.1}$$

2. SwiGLU Activation (Shazeer, 2020): Replaced standard ReLU FFN with SwiGLU:

$$\text{FFN}_{\text{SwiGLU}}(\mathbf{x}) = (\text{SiLU}(\mathbf{x}\mathbf{W}_{\text{gate}}) \odot \mathbf{x}\mathbf{W}_{\text{up}})\mathbf{W}_{\text{down}} \tag{4.2}$$

Uses $\frac{2}{3} \cdot 4d = \frac{8}{3}d_{\text{model}}$ for the intermediate dimension to keep parameter count comparable.

3. Rotary Positional Embeddings (RoPE, Su et al., 2021): Replaced absolute position embeddings. Encodes position via rotation of Q and K vectors in 2D subspaces. Enables natural relative position modeling.

4.1.3 Training

- **Sizes:** 7B, 13B, 33B, 65B. Trained on 1.0–1.4T tokens.
- **Data:** CommonCrawl (67%), C4 (15%), GitHub, Wikipedia, Books, ArXiv, StackExchange.
- **Chinchilla-optimal:** LLaMA was trained ∼5% beyond Chinchilla-optimal ratio, prioritizing inference efficiency over training efficiency.
- **Optimizer:** AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.95$, cosine LR schedule, 2000-step warmup.
- **Efficient implementation:** xformers (Rabe & Staats for memory-efficient attention), reduced activation recomputation, manual backward instead of autograd for some operations.

4.1.4 Results

- LLaMA-13B outperformed GPT-3 (175B) on most benchmarks.
- LLaMA-33B and 65B competitive with PaLM (540B).
- **Key claim:** "Open models can match or exceed closed models when trained on public data with careful architecture design."

4.2 LLaMA 2: Open Foundation and Fine-Tuned Chat Models (Touvron et al., 2023)

4.2.1 The Paper

Paper: Touvron, H., Martin, L., Stone, K., *et al.* (2023). *Llama 2: Open Foundation and Fine-Tuned Chat Models.*

Released: July 2023. Sizes: 7B, 13B, 70B. Both base models AND chat fine-tuned versions.

4.2.2 Architectural Changes from LLaMA 1

1. Grouped Query Attention (GQA)—70B only: The 70B model introduced GQA (Ainslie et al., 2023), where 8 KV heads serve 64 query heads ($h = 64$, $g = 8$). This dramatically reduces KV-cache memory:

$$\text{KV-cache}_{\text{GQA}} = \text{KV-cache}_{\text{MHA}} \times \frac{g}{h} = \text{KV-cache}_{\text{MHA}} \times \frac{1}{8} \tag{4.3}$$

The 7B and 13B models use standard MHA (MQA was found to degrade quality at smaller scales).

2. Extended Context Length: 4K tokens (vs 2K for LLaMA 1). Achieved through continued pre-training with longer sequences and RoPE frequency adjustments.

3. Training Data: Increased to 2T tokens (vs 1.4T for LLaMA 1). More diverse data mixture with up-sampled factual sources.

4.2.3 Chat Model Training (The Alignment Pipeline)

LLaMA-2-Chat introduced a multi-stage alignment process:

Stage 1—Supervised Fine-Tuning (SFT): Publicly available instruction data (no proprietary GPT-4 outputs). Emphasis on helpfulness and safety.

Stage 2—RLHF with Two Reward Models:

- **Helpfulness RM:** Trained on human preference data ranking model responses.
- **Safety RM:** Separate model trained on adversarial and safety-critical prompts.
- During PPO optimization, the RLHF objective uses whichever RM's score is lower (pessimistic approach—prioritizing safety).

Stage 3—Ghost Attention (GAtt): A technique to reinforce system prompts across multi-turn conversations. The system message is prepended to EVERY user turn during training, ensuring the model doesn't forget instructions across turns.

Stage 4—Iterative Refinement: 5 rounds of RLHF with weekly human preference data collection. Each round improves on the previous. The final model uses Rejection Sampling (generate multiple responses, pick best per RM) for additional improvement.

4.2.4 Results

- LLaMA-2 70B matched or exceeded GPT-3.5 on several benchmarks.
- Chat models rated comparable to ChatGPT in human evaluations.
- **Ghost Attention:** Maintains persona/instructions across multi-turn conversations.

4.3 LLaMA 3 and 3.1: The Big Leap (Meta, April–July 2024)

4.3.1 LLaMA 3 (April 2024) — The Paper

Paper: Meta AI (2024). *The Llama 3 Herd of Models.*

4.3.2 Architectural Changes from LLaMA 2

1. All Models Use GQA: Even the 8B model uses $g = 8$ KV heads. This eliminates the scale-dependent GQA/MHA distinction.

2. Vocabulary Expansion: 128K tokens (vs 32K in LLaMA 1/2). Uses a BPE tokenizer from the tiktoken library (same family as GPT-4's). The larger vocabulary reduces sequence length for the same text, improving throughput and reducing positional confusion.

3. RoPE Frequency Increase: The base RoPE frequency increased from 10,000 to 500,000. Higher base frequency means:

$$\theta_i = 500000^{-2i/d} \quad (\text{vs } 10000^{-2i/d} \text{ in LLaMA 1/2}) \tag{4.4}$$

This improves the model's ability to distinguish nearby positions (better local context modeling).

4. Training Data Scale: 15T+ tokens. 4× more than LLaMA 2.

4.3.3 LLaMA 3.1 (July 2024)

Sizes: 8B, 70B, 405B (the first frontier-class open-weight model).

Key 3.1 innovations:

- 405B uses a more complex architecture with 126 layers, $d = 16384$, 128 attention heads, and specialized parallelism strategies.
- **Long context:** 128K context window via continued pre-training with RoPE frequency scaling (NTK-aware interpolation).
- **Multilingual improvements:** Significant investment in non-English data. Uses temperature-based sampling during pre-training for balanced language representation.
- **Tool use capability:** Native function calling support. The model can generate structured JSON for API calls.
- **Training stability:** 405B model had no irrecoverable loss spikes during training—an engineering achievement for the scale.

4.3.4 Training Recipe (LLaMA 3)

- **Pre-training:** 3 stages of data mixing. Stage 1 (0–2T): general web. Stage 2 (2T–8T): increased code and math. Stage 3 (8T–15T): annealed with higher-quality data, decaying LR.
- **Post-training (alignment):** 4 rounds. Round 1: SFT on human annotations. Round 2: SFT on synthetic data augmented via rejection sampling. Round 3: DPO on preference data. Round 4: Rejection sampling with best-of-N generation.
- **DPO over PPO:** LLaMA 3 uses Direct Preference Optimization (DPO) instead of RLHF PPO for efficiency and stability. The DPO loss:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(x,y_w,y_l)}[\log \sigma(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)})] \tag{4.5}$$

4.3.5 LLaMA 4 (2025)

Announced as MoE-based architecture. Scout (17B active, 109B total) and Maverick (17B active, 400B total) variants. Scout uses 16 experts with 1 activated, Maverick uses 128 experts with 1 activated + 1 shared. Behemoth (288B active, $\sim$2T total) training ongoing as of April 2025. All models are natively multimodal with early fusion (MoE layers process interleaved text and vision tokens).

4.4 LLaMA Architectural Evolution Summary

Table 4.1: LLaMA family architectural evolution

Feature	**LLaMA 1**	**LLaMA 2**	**LLaMA 3**	**3.1 405B**	**LLaMA 4**
Max params	65B	70B	70B (initial)	405B	400B (MoE)
Context	2K	4K	8K	128K	1M+
Attention	MHA	MHA/GQA	GQA all	GQA	GQA
Vocab size	32K	32K	128K	128K	∼200K
RoPE θ	10000	10000	500000	500000	500000
Training tokens	1.4T	2T	15T	15T	30T+
Alignment	None	RLHF (PPO)	DPO	DPO	RL+DPO
Multimodal	No	No	No	No	Yes
MoE	No	No	No	No	Yes

Chapter 5

Mistral: Efficient Open-Weight Models from Europe

Mistral AI emerged as a strong European competitor, contributing architectural innovations for efficient inference.

5.1 Mistral 7B (Jiang et al., 2023)

5.1.1 The Paper

Paper: Jiang, A.Q., Sablayrolles, A., Mensch, A., *et al.* (2023). *Mistral 7B.*

Problem: How to make a 7B model competitive with much larger models?

5.1.2 Architectural Innovations

1. Sliding Window Attention (SWA): Window size $W = 4096$. Each token attends only to the W most recent tokens:

$$\text{Attention}_{\text{SWA}}(\mathbf{q}_i, \mathbf{K}, \mathbf{V}) = \frac{\sum_{j=\max(0,i-W+1)}^{i} \exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d}) \mathbf{v}_j}{\sum_{j=\max(0,i-W+1)}^{i} \exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d})} \tag{5.1}$$

The **receptive field** grows with depth: layer ℓ can access information from $\sim \ell \times W$ tokens away through the layer stack (each layer passes information one window forward). This means theoretically unbounded context—in practice, effective context is ∼16K–32K.

2. Rolling Buffer KV-Cache: Only the last W key-value pairs are cached (a fixed-size circular buffer). This gives **constant** KV-cache memory regardless of sequence length, enabling efficient long-context inference.

3. Grouped Query Attention (GQA): $h = 32$ query heads, $g = 8$ KV heads. Further reduces KV-cache.

4. Pre-fill and Chunking: A custom block pre-fill technique processes long prompts efficiently by chunking them into blocks of size W.

5.1.3 Training

- Trained on an undisclosed dataset (estimated ∼2–3T tokens).
- 32K vocabulary. 4096 hidden dimension, 32 layers.
- Competitive with LLaMA-2 13B despite being nearly half the size.

5.1.4 Mistral 7B Instruct

Fine-tuned on publicly available instruction datasets. Demonstrated that even a 7B model can achieve strong instruction-following without RLHF.

5.2 Mixtral 8×7B: Sparse Mixture of Experts (Jiang et al., 2024)

5.2.1 The Paper

Paper: Jiang, A.Q., Sablayrolles, A., Roux, A., *et al.* (2024). *Mixtral of Experts.*

5.2.2 Architecture

- **8 experts** (each similar to Mistral 7B's FFN), **2 active** per token ($k = 2$).
- **Total:** 46.7B parameters. **Active per token:** 12.9B.
- MoE layers replace the FFN in **alternating** layers (every other layer). Dense attention layers between MoE layers maintain representational continuity.
- Router: $\mathbf{g} = \text{softmax}(\text{TopK}(\mathbf{x}\mathbf{W}_g, k = 2))$.
- Router dimension: input is the token's hidden state (no auxiliary information).

5.2.3 Key Design Decisions

- **Why $k = 2$?** Two experts balance specialization vs robustness. $k = 1$ (Switch) can cause routing collapse; $k > 2$ increases compute without proportional quality gains.
- **Why interleave dense and MoE layers?** Dense attention layers maintain a shared representation space; MoE FFN layers provide specialized processing. This hybrid prevents the model from "fragmenting" into separate sub-models.
- **No auxiliary loss for load balancing:** Mixtral does NOT use a load-balancing auxiliary loss. Instead, it relies on the natural balance of the softmax routing (empirically, load balances without explicit intervention).

5.2.4 Results

- Outperforms LLaMA-2 70B on most benchmarks with 5× fewer active parameters.
- Matches or exceeds GPT-3.5 on several tasks.
- Inference throughput of 12.9B model at 46.7B quality—the value proposition of MoE.

5.3 Mistral Large, Nemo, and Codestral

5.3.1 Mistral Large (2024)

Flagship dense model. Top-tier reasoning and multilingual performance. Proprietary weights. Demonstrates that dense models at scale can still compete with MoE.

5.3.2 Mistral Nemo 12B (July 2024)

A 12B model jointly developed with NVIDIA, trained on DGX Cloud. Uses **Tekken tokenizer**, a BPE tokenizer optimized for 100+ languages with ∼130K vocabulary. 128K context window. Released under Apache 2.0.

5.3.3 Codestral 22B (May 2024)

Code-specialized model. Trained on 80+ programming languages. Uses **fill-in-the-middle (FIM)** training, where the model learns to complete code given surrounding context:

```
# FIM format:
# <prefix>code before cursor<SUFFIX>code after cursor<MIDDLE>
# Model generates the middle part
```

5.3.4 Pixtral 12B (September 2024)

Multimodal variant based on Nemo 12B. Uses a vision encoder (likely ViT-based) projecting image tokens into the language space. Native interleaved image-text understanding.

5.3.5 Mistral Small 3 (2025)

Efficient sub-7B model optimized for low-latency applications. Apache 2.0. Matches larger models on agentic and function-calling tasks.

5.4 Mistral's Design Philosophy

Mistral's consistent design choices:

- **Efficiency-first:** Sliding window + GQA + MoE—every architectural choice prioritizes inference efficiency.
- **Simplicity:** Minimal training tricks. No RLHF in Instruct models (SFT only). The architecture "just works."
- **Open progressive release:** Start with Apache 2.0 weights, gradually release larger/better models, some proprietary.
- **Multilingual by design:** Training data and tokenizers built for multilingual support from the start.

5.5 Mistral Family Summary

Table 5.1: Mistral model family

Model	Date	Params	Active	Context	License
Mistral 7B	Sep 2023	7.3B	7.3B	8K (effective 32K)	Apache 2.0
Mixtral 8×7B	Dec 2023	46.7B	12.9B	32K	Apache 2.0
Mistral Large	Feb 2024	Unk. (dense)	Full	32K	Proprietary
Codestral 22B	May 2024	22B	22B	32K	MNPL
Nemo 12B	Jul 2024	12B	12B	128K	Apache 2.0
Pixtral 12B	Sep 2024	12B	12B	128K	Apache 2.0
Mistral Small 3	Jan 2025	<7B	Full	32K	Apache 2.0

Chapter 6

Qwen Series: Alibaba's Multilingual Powerhouse

Alibaba's Qwen series demonstrates systematic scaling across sizes, languages, and modalities.

6.1 Qwen (Tongyi Qianwen) 1.0 (September 2023)

6.1.1 The Paper

Paper: Bai, J., Bai, S., Chu, Y., *et al.* (2023). *Qwen Technical Report.* arXiv:2309.16609.

6.1.2 Architecture

- **Modified LLaMA architecture:** RMSNorm, SwiGLU, RoPE, no bias terms (like LLaMA).
- **Untied embeddings:** Separate input and output embeddings (vs LLaMA's tied embeddings). Slightly more parameters but better performance for multilingual tasks.
- **Log-N attention scaling:** Instead of LLaMA's $\sqrt{d}$ scaling, Qwen uses $\log_n$ scaling:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\log_n}\right)\mathbf{V} \tag{6.1}$$

  where n is the context position. This ensures attention entropy grows logarithmically with context length.
- **NTK-aware interpolation** for extended context.
- **Vocabulary:** 152K tokens (Chinese-optimized BPE with large CJK coverage).
- **Sizes:** 1.8B, 7B, 14B, 72B.

6.1.3 Training

- 3T tokens. Emphasis on Chinese + English + 27 languages.
- **Two-stage pre-training:** General web (2.2T) → High-quality (0.8T).
- **Alignment:** SFT → RM → PPO (standard RLHF pipeline).

6.1.4 Qwen-Chat

RLHF-trained chat variant. Competitive with LLaMA-2-Chat.

6.2 Qwen 1.5 (February 2024)

6.2.1 Changes from 1.0

- Sizes: 0.5B, 1.8B, 4B, 7B, 14B, 32B, 72B, 110B (dense).
- **GQA for all sizes above 4B.**
- **SwiGLU with bias:** Added bias terms to SwiGLU layers (departing from LLaMA's no-bias design).
- **Extended training:** More tokens, better data mixture.
- **Qwen 1.5-Chat:** Improved instruction-following.

6.3 Qwen2 (June 2024)

6.3.1 The Paper

Paper: Yang, A., Yang, B., *et al.* (Qwen Team, 2024). *Qwen2 Technical Report.*

6.3.2 Architectural Refinements

- Sizes: 0.5B, 1.5B, 7B, 57B (MoE variant with 14B active), 72B.
- **GQA for all sizes** (including 0.5B).
- **Tied embeddings:** Switched to tied input-output embeddings (like LLaMA).
- **Dual chunk attention + YaRN** for long context (32K).
- **Training:** 7T+ tokens. Quality filtering via model-based scoring (Qwen 72B itself scored training data quality).
- **Multilingual:** Expanded to 27 languages. Used contrastive learning to align cross-lingual representations.

6.3.3 Qwen2-57B-A14B (MoE)

First MoE model in the Qwen series. 57B total, 14B active. 64 experts, top-8 activated. Shared experts (4) + routed experts (60). Fine-grained segmentation inspired by DeepSeekMoE.

6.3.4 Results

- Qwen2-72B matches LLaMA-3 70B across benchmarks.
- Strong Chinese performance naturally; competitive English performance.
- 7B model outperforms LLaMA-2 13B and Mistral 7B.

6.4 Qwen2.5 (September 2024)

6.4.1 Key Improvements

- **Data:** 18T tokens. Significant increase in code (from GitHub) and math (from ArXiv, textbooks).
- **Training stability:** Trained without any loss spike or restart.
- **Extended context:** 32K base; 128K via YaRN interpolation.
- **Qwen2.5-Coder:** Code-specialized variant with FIM training, repository-level context.
- **Qwen2.5-Math:** Math-specialized variant with chain-of-thought pre-training + RL-based reasoning enhancement.

6.4.2 Results

- Qwen2.5-72B-Instruct: 86.1 MMLU, competitive with GPT-4 on several categories.
- Qwen2.5-Coder-7B: Outperforms many 30B+ code models.
- Qwen2.5-Math-72B: 90.0 on MATH (near o1-mini level).

6.5 Qwen2.5-VL and Qwen-Audio

- **Qwen-VL:** Vision-language models based on Qwen + ViT. Dynamic resolution, interleaved image-text understanding.
- **Qwen2.5-VL (January 2025):** Enhanced with dynamic frame sampling for video understanding, OCR improvements.
- **Qwen-Audio:** Audio understanding variant.
- **Qwen-Agent:** Agent framework using Qwen models for tool use, planning, and multi-step reasoning.

6.6 Qwen Family Summary

Table 6.1: Qwen architectural evolution

Version	Date	Max Size	Key Changes
Qwen 1.0	Sep 2023	72B (dense)	Modified LLaMA, untied embeds, NTK-aware
Qwen 1.5	Feb 2024	110B (dense)	GQA for >4B, SwiGLU with bias
Qwen2	Jun 2024	72B + 57B MoE	Tied embeds, dual chunk attn, 7T tokens
Qwen2.5	Sep 2024	72B (dense)	18T tokens, code/math specialization
Qwen2.5-VL	Jan 2025	72B	Dynamic frame sampling, video

Chapter 7

Gemma and Phi: Small But Mighty

Google's Gemma and Microsoft's Phi represent two philosophies for building capable small models.

7.1 Gemma: Google's Open Model Family (February 2024)

7.1.1 The Paper

Paper: Gemma Team, Google DeepMind (2024). *Gemma: Open Models Based on Gemini Research and Technology.*

7.1.2 Architecture

Gemma 1.0 (2B, 7B):

- Based on Gemini research but independently trained.
- **GeGLU activation:** Uses GELU gating instead of SiLU in the gated FFN:

$$\text{FFN}_{\text{GeGLU}}(\mathbf{x}) = (\text{GELU}(\mathbf{x}\mathbf{W}_{\text{gate}}) \odot \mathbf{x}\mathbf{W}_{\text{up}})\mathbf{W}_{\text{down}} \tag{7.1}$$

- **Multi-Query Attention (MQA):** All heads share a single KV pair (like PaLM). More aggressive than GQA.
- **Pre-norm RMSNorm.**
- **RoPE positional embeddings.**
- **Vocabulary:** 256K tokens (large vocabulary for multilingual coverage).
- **Context:** 8192 tokens.

7.1.3 Training

- **Data:** 6T tokens for 7B (3T for 2B). Mixture of web documents, code, and math.
- **Distillation from larger models:** Gemma incorporates knowledge distillation in its training (not purely next-token prediction).
- **Instruction tuning:** RLHF, then distilled to Gemma Instruct variants.

7.1.4 Gemma 2 (June 2024)

Sizes: 9B and 27B. Key architectural changes:

1. Interleaving Local-Global Attention: Alternating attention patterns across layers: even layers use sliding window attention (local context, W=4096), odd layers use global attention (full sequence). This reduces compute from $O(n^2)$ to $\sim O(n \cdot n/2 + n \cdot W)$.

2. Logit Soft-Capping: A novel regularization: cap logits at a maximum value using a soft capping function:

$$\text{logit}_{\text{capped}} = \text{c} \cdot \tanh(\text{logit}/\text{c}) \tag{7.2}$$

where c = 30 for Gemma 2 27B and c = 50 for 9B. This prevents the model from becoming overconfident and stabilizes training (prevents logit explosion during RLHF).

3. Pre-Norm and Post-Norm Combination: Uses both pre-layer norm AND post-attention norm (a departure from pure pre-norm). This sandwich normalization improves training stability.

4. RMSNorm without learnable parameters: Simplified RMSNorm: drops the learnable $\boldsymbol{\gamma}$. Slightly fewer parameters.

5. Grouped Query Attention (GQA): Switched from MQA to GQA. Better quality for the parameter count.

7.1.5 Results

- Gemma 2 27B outperforms LLaMA-3 70B on LMSys Chatbot Arena while being 2.6× smaller.
- Gemma 2 9B competitive with LLaMA-3 8B and Mistral 7B.
- Demonstrates that architectural innovation (local-global attn, soft-capping, sandwich norm) can compensate for smaller scale.

7.1.6 RecurrentGemma (April 2024)

An RNN-based variant using the Griffin architecture (RG-LRU). Linear complexity in sequence length. Optimized for very long context and edge deployment with constant memory.

7.1.7 CodeGemma and ShieldGemma (2024)

CodeGemma: Fine-tuned on code. ShieldGemma: Safety classifier model (not generative, outputs safety scores).

7.2 Phi Series: Data Quality Over Scale (Microsoft)

7.2.1 Phi-1: "Textbooks Are All You Need" (Gunasekar et al., June 2023)

Paper: Gunasekar, S., Zhang, Y., Aneja, J., *et al.* (2023). *Textbooks Are All You Need.*

Philosophy: The quality of training data matters more than its quantity.

Architecture: 1.3B parameters. Standard decoder-only transformer. Trained on **synthetic textbook-quality data**:

- Generated 1B tokens of Python code textbooks and exercises using GPT-3.5.
- Filtered for quality, correctness, and educational value.
- **Result:** 50.6% on HumanEval (better than most 10B+ models at the time). Surpassed StarCoder 15B on code tasks.

7.2.2 Phi-1.5 (September 2023)

Extended to natural language reasoning. Same 1.3B architecture. Training data expanded to include synthetic NL reasoning data (math, logic, common sense). Demonstrated that synthetic data quality can drive reasoning in small models.

7.2.3 Phi-2 (December 2023)

2.7B parameters. Two key innovations:

- **Data pruning:** Used a classifier to score training examples for "educational value" and kept only the top-scoring data. This is essentially model-based data filtering at scale.
- **Synthetic augmentation:** Generated additional training data from larger models, then filtered for quality.
- **Result:** Matched 7B models on reasoning benchmarks. Showed that 2.7B can outperform 7B with better data.

7.2.4 Phi-3 (April–August 2024)

Paper: Abdin, M., Jacobs, S.A., Awan, A.A., *et al.* (2024). *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone.*

Architecture:

- **Phi-3-Mini:** 3.8B parameters. 32 layers, 3072 hidden dim. Context: 4K and 128K variants.
- **Phi-3-Small:** 7B parameters. Standard LLaMA-like architecture.
- **Phi-3-Medium:** 14B.
- **All use:** GQA, SwiGLU, RMSNorm, RoPE. Tiktoken tokenizer (same tokenizer family as GPT-4 and LLaMA-3).

Training Data Philosophy (Phi-3's "data curriculum"):

1. **Phase 1—Web data:** Standard web crawl, heavily filtered (only 5–10% of CommonCrawl kept).
2. **Phase 2—Synthetic data:** Large models generate "reasoning traces" (step-by-step solutions to math, coding, logic problems).
3. **Phase 3—High-quality curation:** Books, academic papers, curated Q&A.
4. **Result:** "Data optimal regime" where small models benefit disproportionately from high-quality data.

7.2.5 Phi-3-Vision

Multimodal extension with a vision encoder (CLIP-style). 4.2B total parameters. Demonstrates multimodal reasoning at phone-deployable scale.

7.2.6 Phi-3.5 (August 2024)

MoE variant: 6.6B total, 3.8B active (Phi-3-Mini × 2 experts, top-1 activated + shared expert). Multi-lingual support expanded (Arabic, Chinese, etc.). 128K context.

7.2.7 Phi-4 (December 2024)

14B parameters. Further refinement of the data-centric approach. Introduces **execution feedback** for code data (generate code, execute it, include execution results in training). Achieves state-of-the-art for sub-20B models on reasoning tasks.

7.2.8 The Phi Philosophy Summary

- **Data quality » data quantity** for small models.
- **Synthetic data generation** from larger models creates "textbooks" for training.
- **The model can teach itself:** Use generations from the same model after alignment to improve the base model.
- **Execution feedback:** Generate, execute, verify—include results in the training loop.
- **Key insight:** A 3.8B model with Phi-quality data can match a 7B model with standard data. The data advantage compounds at small scales.

7.3 Small Model Comparison

Table 7.1: Gemma vs Phi design philosophies

Aspect	Gemma	Phi
Origin	Distilled from Gemini	Trained from scratch with synthetic data
Architecture	GeGLU, MQA/GQA, local-global attn	GQA, SwiGLU, standard decoder
Key innovation	Attention patterns, soft-capping, norms	Synthetic data curriculum, data pruning
Unique features	RecurrentGemma (RNN), ShieldGemma	Execution feedback, MoE at small scale
Tokenizer	256K (large, multilingual)	100K (tiktoken, GPT-4 compatible)
Best model	Gemma 2 27B	Phi-4 14B

Chapter 8

The DeepSeek Family: A Complete Technical History

DeepSeek exemplifies systematic, paper-by-paper architectural progression. Each paper builds on the last, with clear problem statements and innovations. This chapter traces the full lineage.

8.1 Paper #1: DeepSeek LLM—Scaling Open-Source Language Models (Jan 2024)

8.1.1 The Paper

Paper: DeepSeek-AI (2024). *DeepSeek LLM: Scaling Open-Source Language Models with Longtermism.* arXiv:2401.02954.

Problem: Open-source LLMs lagged behind closed models (GPT-4) in both scale and capability. Previous open models (LLaMA 2, Mistral) capped at 70B. How to scale open LLMs to truly competitive levels?

8.1.2 Scale and Architecture

- **Parameters:** 7B and 67B variants (a clear message: matching LLaMA 2's sizes).
- **Architecture:** Largely follows LLaMA design—Pre-norm RMSNorm, SwiGLU activation, RoPE.
- **Training tokens:** 2T tokens for both models. Trained on a carefully curated mixture of web text, code, math, and books.
- **Training data:** DeepSeek curated their own dataset emphasizing Chinese + English. Significant deduplication and quality filtering.

8.1.3 Key Innovation: Scaling Laws for Data

DeepSeek studied **data scaling laws** specifically. Finding: models benefit from more training tokens even beyond Chinchilla-optimal ratios. This "longtermism" philosophy—train on as much data as possible—became a hallmark of DeepSeek's approach:

- The 67B model was trained on 2T tokens (vs ∼1.4T Chinchilla-optimal). Shows continued improvement with more data.

- This finding influenced DeepSeek-V2 and V3's massive data scales.

8.1.4 Results

- DeepSeek 67B matched or exceeded LLaMA-2 70B across all benchmarks.
- Strong performance in both Chinese and English.
- Code and math performance notably strong from the beginning.

8.2 Paper #2: DeepSeekMoE—Towards Ultimate Expert Specialization (Jan 2024)

8.2.1 The Paper

Paper: Dai, D., *et al.* (DeepSeek-AI, 2024). *DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models.* arXiv:2401.06066.

Problem: Conventional MoE (GShard, Switch Transformer) suffers from **knowledge redundancy** (multiple experts learn overlapping knowledge) and **knowledge hybridity** (individual experts handle too many unrelated types of knowledge).

8.2.2 DeepSeekMoE Architecture

Two types of experts:

1. Shared Experts (always activated): A small number of experts (K_s, typically 1–2) that process EVERY token. These capture common knowledge needed by all inputs (grammar, general semantics, common patterns):

$$\mathbf{y}_{\text{shared}} = \sum_{i=1}^{K_s} \text{FFN}_i^{\text{shared}}(\mathbf{x}) \tag{8.1}$$

2. Routed Experts (selectively activated): A larger number of experts (K_r, e.g., 64), only top-k activated per token. These handle specialized knowledge:

$$\mathbf{y}_{\text{routed}} = \sum_{i \in \text{TopK}(\mathbf{x}\mathbf{W}_g, k)} g_i \cdot \text{FFN}_i^{\text{routed}}(\mathbf{x}) \tag{8.2}$$

Final output:

$$\mathbf{y} = \mathbf{y}_{\text{shared}} + \mathbf{y}_{\text{routed}} \tag{8.3}$$

8.2.3 Fine-Grained Expert Segmentation

DeepSeekMoE splits each standard FFN into **smaller** experts. Instead of E large experts, use mE experts each with $1/m$ the hidden dimension:

- Standard MoE: 8 experts, hidden size $4d$.
- Fine-grained MoE: 64 experts, hidden size $d/2$ (or similar). $m = 8\times$ more experts, each smaller.

This enables more **fine-grained knowledge specialization**—each expert becomes a narrow specialist. The router can compose multiple fine-grained experts for a token, achieving higher expressivity.

8.2.4 Load Balancing

Two auxiliary losses:

- **Expert-level balance loss:** Encourages uniform routing across experts at the global level.
- **Device-level balance loss:** When experts are distributed across GPUs, encourages uniform routing across *devices* to minimize all-to-all communication imbalance.

8.2.5 Key Results

- DeepSeekMoE 16B (active) matches DeepSeek 7B Dense performance with 40% fewer FLOPs.
- Scaling to 145B total (22B active) outperforms GShard-style MoE at equal active params.
- Fine-grained segmentation + shared experts = the DeepSeekMoE formula.

8.3 Paper #3: DeepSeek-V2—Strong, Economical, and Efficient (May 2024)

8.3.1 The Paper

Paper: DeepSeek-AI (2024). *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.* arXiv:2405.04434.

Problem: How to achieve GPT-4-class performance at a fraction of the cost? DeepSeek-V2 aimed to be both powerful AND cheap to run, introducing two critical innovations.

8.3.2 Innovation 1: Multi-Head Latent Attention (MLA)

The KV-Cache Problem: Standard MHA requires caching keys and values for every head at every layer. For a model with L layers, h heads, d_h dimensions, and S tokens, this costs $2LShd_h$ elements in FP16.

MLA Solution: Compress keys and values into a low-rank **latent** representation, then reconstruct per-head K and V only during attention computation:

$$\mathbf{c}_t^{KV} = \mathbf{x}_t \mathbf{W}^{DKV} \in \mathbb{R}^{d_c} \quad \text{(compressed latent, } d_c \ll d\text{)} \tag{8.4}$$

$$\mathbf{k}_t^{(i)} = \mathbf{c}_t^{KV} \mathbf{W}_i^{UK} \quad \text{(reconstruct per-head K)} \tag{8.5}$$

$$\mathbf{v}_t^{(i)} = \mathbf{c}_t^{KV} \mathbf{W}_i^{UV} \quad \text{(reconstruct per-head V)} \tag{8.6}$$

The KV-cache stores only $\mathbf{c}_t^{KV}$, not the full $\{\mathbf{k}_t^{(i)}, \mathbf{v}_t^{(i)}\}$. Memory saving: $d/(h \cdot d_c \cdot 2)$.

Query Compression: Queries are also compressed:

$$\mathbf{c}_t^{Q} = \mathbf{x}_t \mathbf{W}^{DQ} \in \mathbb{R}^{d_c}, \quad \mathbf{q}_t^{(i)} = \mathbf{c}_t^{Q} \mathbf{W}_i^{UQ} \tag{8.7}$$

This reduces activation memory during training (no need to materialize full Q for all heads before attention).

RoPE in MLA: RoPE is applied *after* decompression for queries and *split* for keys (part of K gets RoPE, part doesn't). The decompressed K and Q have a portion dedicated

to positional encoding via RoPE:

$$\mathbf{k}_t^{(i)} = [\mathbf{k}_{t,\text{nope}}^{(i)};\ \text{RoPE}(\mathbf{k}_{t,\text{pe}})] \tag{8.8}$$

where $\mathbf{k}_{t,\text{nope}}$ carries content information and $\mathbf{k}_{t,\text{pe}}$ carries position information (only a small fraction of dimensions).

8.3.3 Innovation 2: DeepSeekMoE at Scale

DeepSeek-V2 scaled the DeepSeekMoE architecture:

- Total params: 236B. Active params: 21B (only 8.9% activated!).
- $K_s = 2$ shared experts, $K_r = 160$ routed experts. Top-6 activated.
- $m = 2.5\times$ fine-grained segmentation.
- Expert hidden dimension: 1536 (vs 11008 for a dense variant).

8.3.4 Training and Cost

- Trained on 8.1T tokens.
- **Training cost:** ∼5M USD (economical by design).
- **Inference cost:** ∼1/5–1/10 of comparable dense models (only 21B active parameters).
- **Alignment:** SFT + GRPO (Group Relative Policy Optimization). No PPO. The simple GRPO approach: sample multiple responses per prompt, rank them, use the reward signal to apply DPO-style optimization.

8.3.5 Results

- Matched or exceeded GPT-4 on Chinese benchmarks.
- Competitive with LLaMA-3 70B on English benchmarks with 1/3 active params.
- API pricing: ∼1/10–1/20 of GPT-4 Turbo per token.
- **Key claim:** "Strongest open-source model yet" at release.

8.4 Paper #4: DeepSeek-Coder-V2 (June 2024)

8.4.1 The Paper

Paper: Zhu, Q., *et al.* (DeepSeek-AI, 2024). *DeepSeek-Coder-V2: Breaking the Barrier of Closed-Source Models in Code Intelligence.* arXiv:2406.11931.

8.4.2 Architecture

- Based on DeepSeek-V2 architecture (MLA + DeepSeekMoE).
- 236B total, 21B active.
- **Additional pre-training on code:** 6T tokens of code from GitHub, StackOverflow, code documentation. Total pre-training mix: 60% code, 10% math, 20% natural language.

- **Fill-in-the-Middle (FIM):** Uses the PSM (Prefix-Suffix-Middle) format during pre-training:

  <fim_prefix>code_before<fim_suffix>code_after<fim_middle>code_middle<eos> (8.9)

  This teaches the model to complete code given surrounding context.
- Supports 338 programming languages.

8.4.3 Results

- Matches GPT-4 Turbo on HumanEval, MBPP, LiveCodeBench.
- First open-source model competitive with GPT-4 on code tasks.
- 128K context window, enabling repository-level code understanding.

8.5 Paper #5: DeepSeek-V3—The Giant Leap (December 2024)

8.5.1 The Paper

Paper: DeepSeek-AI (2024). *DeepSeek-V3: A 671B MoE Language Model with Multi-Token Prediction and Auxiliary-Loss-Free Load Balancing.* arXiv:2412.19437.

Released: Dec 26, 2024. The model that shocked the AI world.

8.5.2 Architectural Innovations

1. Massive MoE Scale:

- Total params: 671B. Active params: 37B per token (only 5.5%).
- $K_s = 1$ shared expert, $K_r = 256$ routed experts. Top-8 activated per token.
- Each expert is a small FFN: intermediate size ∼2048.
- Fine-grained segmentation: far more experts than V2.

2. Multi-Token Prediction (MTP): Instead of predicting only the next token, V3 predicts the **next** n **tokens** (with $n = 2$ used in practice):

- After the main model, $n-1$ additional sequential prediction modules take the final hidden state and predict subsequent tokens.
- Each module uses a shared output head but has its own transformer block.
- Training loss: $\mathcal{L} = \sum_t \sum_{i=0}^{n-1} \lambda_i \cdot \log p(y_{t+i} \mid y_{<t})$ where $\lambda_0 = 1.0$ and λ_i decays for $i > 0$.
- Benefits: (a) Richer training signal from each token. (b) Improved sample efficiency. (c) At inference, MTP modules can be used for **speculative decoding**—the multi-token predictions serve as draft tokens, verified by the main model.

3. Auxiliary-Loss-Free Load Balancing: Traditional MoE uses auxiliary losses to balance expert usage. V3 introduces a **dynamic bias** approach:

- Each expert i has a learned bias term b_i added to the router logits: $\tilde{g}_i = g_i + b_i$.
- After each training step, if an expert is overloaded (received too many tokens), reduce its bias b_i. If underloaded, increase b_i.
- No gradient from the bias adjustment. Purely heuristic updates.
- This achieves load balancing WITHOUT auxiliary loss gradients interfering with the primary objective.

- Complemented by a **complementary sequence-wise auxiliary loss** for extreme imbalances (rare).

4. FP8 Mixed Precision Training: First large-scale LLM trained predominantly in FP8:

- GEMM operations (most FLOPs): FP8 with E4M3 format (4 exponent, 3 mantissa bits).
- Attention softmax: BF16 (requires higher precision).
- Normalization layers: BF16.
- Embedding: BF16.
- **Block-wise quantization** for activations and weights in FP8 GEMM: divide into 128×128 tiles, compute per-tile scaling factors.
- **Result:** ∼2× faster training than BF16, ∼40% less memory.

8.5.3 Training Details

- **Data:** 14.8T tokens. Curated mixture of web text, code, math, books, and multilingual data.
- **Training framework:** HAI-LLM framework with custom CUDA kernels, 2048 NVIDIA H800 GPUs.
- **Training duration:** ∼2 months. Total cost: ∼5.576M USD (remarkably low for a 671B model).
- **Two-stage training:** Stage 1 (12.8T tokens): general pre-training. Stage 2 (2T tokens): extended context to 128K via continued training with longer sequences and YaRN RoPE interpolation.
- **Optimizer:** AdamW, $\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay 0.1, gradient clipping 1.0.
- **LR schedule:** 2000-step warmup to 2.4×10^{-4}, then cosine decay to 2.4×10^{-5}. In the final 10% of training, LR is further decayed to 1/10 for annealing.

8.5.4 Alignment (Post-Training)

- **SFT:** 1.5M high-quality instruction examples (human-curated + DeepSeek-R1 generated reasoning traces).
- **GRPO (Group Relative Policy Optimization):** For RL-based alignment. For each prompt, sample G responses (typically 4–16). Score them. Use the group's mean reward as baseline:

$$\mathcal{L}_{\text{GRPO}} = -\mathbb{E}_{(x,\{(y_i,r_i)\}_{i=1}^{G})}[\max(y_i \text{ with } r_i > \bar{r})] \tag{8.10}$$

  Combined with DPO-style training on preference pairs.
- **Reward signals:** Rule-based rewards for math (final answer match) and code (compilation + test pass). RM-based rewards for open-ended generation.

8.5.5 Results

- Matches or exceeds GPT-4o and Claude 3.5 Sonnet across most benchmarks.
- 87.5% on MATH-500, 90.2% on MMLU.
- 128K context. Strong multilingual (Chinese, English).
- API pricing: ∼1/20–1/50 of GPT-4o per token.
- **Open weights:** MIT license. The most capable fully open model at the time.

8.6 Paper #6: DeepSeek-R1—Reasoning via Reinforcement Learning (January 2025)

8.6.1 The Paper

Paper: DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.* arXiv:2501.12948.

Problem: How to elicit strong reasoning (chain-of-thought) from LLMs without massive supervised datasets of reasoning traces?

8.6.2 DeepSeek-R1-Zero: Pure RL, No SFT

The paper first demonstrates a remarkable experiment: **pure RL on the base model** (DeepSeek-V3-Base), with NO supervised fine-tuning:

- **Starting point:** The raw DeepSeek-V3 base model (no instruction tuning).
- **Training:** GRPO (Group Relative Policy Optimization) with rule-based rewards.
- **Reward design:**
  - **Accuracy rewards:** For math problems with known answers, binary reward for correct final answer. For code (LeetCode), compiler-based binary reward (passes tests or not).
  - **Format rewards:** Reward model for placing reasoning between `<think>` and `</think>` tags, answer between `<answer>` and `</answer>` tags.
- **Prompt template:** Simple instruction asking for step-by-step reasoning:

```
A conversation between User and Assistant. The user asks
a question, and the Assistant solves it. The assistant
first thinks about the reasoning process in the mind and
then provides the user with the answer.
```

Emergent Behaviors During Training: As RL training progressed, the model spontaneously developed sophisticated reasoning behaviors:

- **Self-verification:** The model would state an intermediate conclusion, then check: "Wait, is this correct? Let me verify..."
- **Reflection:** The model would re-evaluate earlier reasoning steps and correct errors.
- **Alternative approaches:** The model would consider multiple solution paths: "Let me try a different approach..."
- The length of reasoning chains naturally increased from a few lines to thousands of tokens as training progressed.
- An "aha moment" (the paper's term) was observed where the model learned to retrace and correct itself mid-generation.

8.6.3 DeepSeek-R1: Cold-Start Data + RL

R1-Zero was impressive but had issues: poor readability, language mixing, and sometimes incoherent chains of thought. R1 fixes this:

Stage 1—Cold Start (Small SFT): Collect thousands of high-quality chain-of-thought examples (using R1-Zero outputs filtered for quality, plus human refinement).

Fine-tune V3-Base on this **cold-start data**. This teaches the model proper reasoning format and readability.

Stage 2—Reasoning-Oriented RL: Apply GRPO to the cold-start model, focusing on reasoning tasks (math, code, science, logic puzzles). Same rule-based rewards as R1-Zero. Add a **language consistency reward** (penalize mixing Chinese and English in the same response).

Stage 3—Rejection Sampling + SFT: Use the RL-trained checkpoint to generate 600K reasoning traces via rejection sampling. Add 200K general (non-reasoning) traces. Fine-tune V3-Base on this combined dataset (800K).

Stage 4—Diverse RL: Final RL stage with rewards spanning helpfulness, harmlessness, AND reasoning accuracy. The reward signal now includes a learned RM for subjective quality + rule-based RM for reasoning.

8.6.4 Distillation to Smaller Models

A crucial contribution: DeepSeek distilled R1's reasoning capabilities into smaller dense models:

- Generated 800K reasoning traces from DeepSeek-R1.
- Fine-tuned Qwen-1.5B, 7B, 14B, 32B and LLaMA-8B, 70B on these traces.
- **Result:** Distilled 7B model outperforms most 70B+ models on reasoning benchmarks.
- **Demonstration:** Reasoning capability can be distilled from large models to small ones—the reasoning patterns learned by R1 transfer effectively.

8.6.5 Key Results

- DeepSeek-R1: 79.8% on AIME 2024 (matching OpenAI o1).
- R1-Zero: 71.0% on AIME (pure RL, no SFT—remarkable).
- Distilled Qwen-32B: 72.6% on AIME (competitive with much larger models).
- Distilled LLaMA-70B: 70.0% on AIME.
- **Key insight:** RL alone (without SFT) can induce sophisticated reasoning behaviors. Cold-start SFT improves readability and format. Distillation effectively transfers reasoning to smaller models.

8.7 Paper #7: DeepSeek-Prover (2024–2025)

A parallel line of work on formal theorem proving in Lean 4:

- **DeepSeek-Prover-V1 (2024):** Fine-tuned DeepSeek-V2 on formal proof data. Generates Lean 4 proofs for IMO/AMC-level problems.
- **DeepSeek-Prover-V2 (2025):** Uses the R1-style RL training to improve proof generation. Combined with an iterative proof search strategy.

8.8 Paper #8: DeepSeek-VL/VL2 (Vision-Language)

Multimodal extensions:

- **DeepSeek-VL (March 2024):** Vision-language model based on DeepSeek LLM + SigLIP/SAM vision encoder.

- **DeepSeek-VL2 (December 2024):** Based on DeepSeek-V2 architecture. Dynamic resolution tiling (not fixed resolution). Mixture of tile encoders for different resolutions. Enhanced visual grounding and OCR capabilities.

8.9 DeepSeek Design Philosophy: A Summary

Table 8.1: DeepSeek paper lineage and key innovations

Paper	Date	Model	Key Innovation
#1 DeepSeek LLM	Jan 2024	7B/67B Dense	Data scaling laws, "longtermism"
#2 DeepSeekMoE	Jan 2024	MoE 16B act.	Shared+routed experts, fine-grained segmentation
#3 DeepSeek-V2	May 2024	MoE 236B/21B	**MLA** (KV compression), economical MoE
#4 DeepSeek-Coder-V2	Jun 2024	MoE 236B/21B	V2 architecture + code FIM training
#5 DeepSeek-V3	Dec 2024	MoE 671B/37B	**MTP**, aux-loss-free LB, FP8 training
#6 DeepSeek-R1	Jan 2025	Reasoning	Pure RL $\rightarrow$ reasoning emergence, cold-start+RL pipeline
#7 Prover	2024–25	Theorem Proving	Formal proof generation with RL
#8 VL/VL2	2024	Multimodal	Dynamic resolution ViT + V2 backbone

Chapter 9

Claude: Anthropic's Safety-First Frontier Models

Anthropic's Claude series prioritizes safety and alignment while pushing the frontier of capability.

9.1 Constitutional AI (Bai et al., December 2022)

9.1.1 The Paper

Paper: Bai, Y., Kadavath, S., Kundu, S., *et al.* (2022). *Constitutional AI: Harmlessness from AI Feedback.*

Problem: RLHF requires human annotators to judge harmful outputs, which is expensive and potentially traumatizing. Can AI provide the feedback instead?

9.1.2 The Constitutional AI Method

Two stages:

Stage 1—Supervised (Critique + Revision):

1. Model generates responses to harmful prompts.
2. The model **critiques its own response** according to a **constitution**—a set of principles (e.g., "Choose the response that is most harmless, helpful, and honest").
3. The model **revises** its response based on the critique.
4. Fine-tune on the revised responses.

Stage 2—RL (AI Feedback):

1. Generate response pairs from the fine-tuned model.
2. AI (another instance of the model) provides preference labels based on the constitution.
3. Train a preference model using these AI-generated preferences.
4. Apply RL (PPO) using the AI preference model as reward.

Result: Produces a model that is more harmless than an RLHF-trained model, without requiring humans to view harmful content. The constitution is transparent and auditable (unlike human annotator guidelines).

9.2 Claude 1 (March 2023)

9.2.1 The Model

Initial release. Built on constitutional AI principles. Key features:

- **100K context window** (unprecedented at release). Achieved via custom attention optimizations.
- **Strong refusal capabilities:** Carefully calibrated to refuse harmful requests while minimizing over-refusal on benign requests.
- **Multi-turn conversation** with extended context memory.

9.3 Claude 2 (July 2023)

9.3.1 Improvements

- Extended to 100K context (same as 1.0 but more refined).
- Improved reasoning, coding, and math.
- 2× longer responses (up to 3–4K tokens per generation).
- Improved factual accuracy. Trained on more up-to-date data.
- **Bar exam:** 76.5% (vs Claude 1's 73.0% and GPT-4's 75.7%).
- **GRE:** 92% percentile verbal, 80% quantitative.

9.4 Claude 3 (March 2024)

9.4.1 The Family

Haiku (fastest), **Sonnet** (balanced), **Opus** (most capable).

9.4.2 Architectural Clues (From Anthropic's Research)

While Anthropic doesn't publish detailed architecture specs, their research papers give clues:

- **Multimodal:** All Claude 3 models accept images (up to ∼20 MB, with text extraction from PDFs, images, etc.).
- **200K context window** (Claude 3); later extended to 1M+ for specific use cases.
- **Thoughtful refusal:** More nuanced than simple refusal. Models distinguish between genuinely harmful requests, borderline cases, and benign requests more accurately.
- Anthropic's research on **mechanistic interpretability** (dictionary learning / sparse autoencoders) and **RLHF refinement techniques** likely influenced Claude 3's training.

9.4.3 Results

- Claude 3 Opus: 86.8% on MMLU (5-shot), 50.4% on GPQA (graduate-level reasoning). Matched or exceeded GPT-4 on several benchmarks.
- Claude 3 Haiku: Competitive with GPT-3.5 despite being much faster/cheaper.

9.5 Claude 3.5 (June 2024)

9.5.1 Claude 3.5 Sonnet

Released as a "mid-tier intelligent, top-tier speed" model. Key advances:

- **Artifact system:** Can generate and render code, documents, and visualizations in a side-by-side interface.
- **Frontier coding:** 92.0% on HumanEval (state-of-the-art at release).
- **Graduate-level reasoning:** 59.4% on GPQA.
- **Vision:** 89.2% on MMMU (val), best-in-class at release.
- **2× faster** than Claude 3 Opus while being more capable.

9.5.2 Claude 3.5 Haiku (October 2024)

Extremely fast, compact model. Competitive with Claude 3 Opus on several benchmarks while being a fraction of the cost.

9.5.3 Computer Use (October 2024)

Anthropic released a Claude variant capable of **controlling a computer**: viewing screenshots, moving the mouse, clicking, and typing. This is not a separate architecture but a specialized fine-tune that generates coordinate-based actions as tokens.

9.5.4 Claude 3.5 Opus (Cancelled?)

Anthropic reportedly trained Claude 3.5 Opus but did not release it, citing that the cost/performance ratio didn't justify deployment vs. Sonnet. This reveals a shift toward efficiency in frontier model development.

9.6 Anthropic's Key Research Papers

9.6.1 Training a Helpful and Harmless Assistant (Bai et al., 2022)

Introduced the HHH (Helpful, Harmless, Honest) framework. Used iterative RLHF with two separate RMs (helpfulness + harmlessness) with a **KL-constrained PPO**.

9.6.2 Towards Monosemanticity: Decomposing Language Models With Dictionary Learning (Dec 2023)

A landmark interpretability paper. Trained sparse autoencoders on Claude's activations and found **interpretable features** at specific layers. Discovered features for concepts like "Golden Gate Bridge," "deception," "inner conflict," etc. Demonstrated that individual neurons are polysemantic, but sparse decompositions can reveal monosemantic features.

9.6.3 Scaling Monosemanticity (May 2024)

Scaled dictionary learning to Claude 3 Sonnet. Extracted millions of interpretable features. Demonstrated **feature steering**: manipulating specific features (e.g., "sycophancy," "deception") can change model behavior predictably.

9.6.4 The Claude Model Card (March 2024/2025)

Anthropic publishes detailed model cards including:

- **Responsible Scaling Policy (RSP):** Formal commitments about when to pause development based on capability thresholds.
- **Alignment stress tests:** Extensive red-teaming for persuasion, deception, CBRN risks.
- **Constitutional principles explicitly listed:** Based on the UN Declaration of Human Rights, Apple Terms of Service, and other public frameworks.

9.7 Anthropic's Philosophy vs. OpenAI

Table 9.1: Anthropic vs OpenAI design philosophies

Aspect	**Anthropic (Claude)**	**OpenAI (GPT)**
Primary focus	Safety, honesty, harmlessness	Capability, general intelligence
Alignment method	Constitutional AI (self-critique + AI feedback)	RLHF (human feedback)
Transparency	Model cards, RSP, constitution public	Minimal architecture disclosure
Interpretability	Heavy investment (sparse autoencoders)	Limited public investment
Release strategy	Staged with safety evaluation	Staged with capability evaluati
Context philosophy	Very long context (200K+)	Moderate context (32–128K)
Multimodality	Image input, text output	Omni-modal (GPT-4o: image/t

Chapter 10

MoE Architectures Beyond DeepSeek

Mixture of Experts models scale capacity without proportional compute increase. This chapter covers major MoE architectures besides DeepSeek.

10.1 DBRX (Databricks, March 2024)

10.1.1 The Paper

Paper: Databricks Mosaic Research Team (2024). *DBRX Technical Blog.*

10.1.2 Architecture

- 132B total, 36B active per token. **16 experts**, top-4 activated ($k = 4$).
- **Fine-grained experts:** Each expert has a smaller hidden dimension than standard FFN. 3.75× smaller than a comparable dense FFN.
- **Rotary Position Embeddings (RoPE)** with 32K max position. Uses GPT-4 tokenizer via tiktoken.
- **Training:** 12T tokens. 3072 NVIDIA H100s over ∼3 months.
- **Megablocks CUDA kernels:** Custom GPU kernels that efficiently handle the sparse MoE forward/backward pass without dropping tokens. Avoids the padding waste of traditional MoE implementations.

10.1.3 Key Innovation: No Token Dropping

DBRX uses **MegaBlocks** (Gale et al., 2022)—a block-sparse kernel that processes MoE layers without the capacity factor / token dropping approach. Each expert processes its assigned tokens directly; the kernel handles variable-length expert inputs efficiently via block-sparse matrix multiplication.

10.1.4 Results

Outperformed GPT-3.5 and was competitive with early Gemini on programming. Released as open-weight under a custom license.

10.2 Jamba (AI21 Labs, March 2024)

10.2.1 The Paper

Paper: AI21 Labs (2024). *Jamba: A Hybrid Transformer-Mamba Language Model.*

10.2.2 Architecture: Mamba + Transformer + MoE

Jamba combines three architectural paradigms in one model:

- **Blocks:** Each Jamba block contains either an attention layer or a Mamba SSM layer, plus an MoE FFN with 16 experts (top-2 activated). Blocks alternate: Attention-MoE → Mamba-MoE → Attention-MoE → ...
- The ratio of attention to Mamba layers is ∼1:7 (few attention layers, many Mamba layers).
- **Attention layers provide:** Precise token-level comparisons (what transformers excel at).
- **Mamba layers provide:** Efficient long-range context compression (what SSMs excel at).
- **MoE layers provide:** Large parameter capacity with low compute.
- **52B total, 12B active.** 256K context. Single 80GB GPU can fit the model at inference.

10.2.3 Key Innovation: Hybrid Architecture

Jamba proves that different sequence mixing paradigms can coexist in a single model. The attention layers handle tasks requiring precise token comparisons; the Mamba layers compress long-range dependencies efficiently. This hybrid design achieves:

- 3× throughput of Mixtral 8×7B at long context.
- Fits in single 80GB GPU via combined SSM efficiency + MoE activation sparsity.

10.3 Qwen2-MoE (Alibaba, June 2024)

- 57B total, 14B active. 64 routed experts (top-8 activated) + 4 shared experts.
- Fine-grained segmentation inspired by DeepSeekMoE.
- **Upcycling:** Initialized from Qwen-1.8B dense weights (replicated and noised). Only 1.1T additional tokens for MoE pre-training.

10.4 Grok-1 (xAI, March 2024)

- 314B total, ∼78B active (estimated). 8 experts, top-2 activated.
- **Architecture:** Standard MoE with 25% of layers being MoE FFN, remaining dense.
- Rotational embeddings (RoPE). 64 layers, 48 heads. Context: 8192.
- **Training:** Trained on a custom dataset including real-time X (Twitter) data.
- **Open-source:** Apache 2.0 weights. One of the largest open-weight MoE models.
- **Grok-2 (August 2024):** Improved version. Closed-source.

10.5 Switch Transformer (Fedus et al., 2022)

Paper: Fedus, W., Zoph, B., & Shazeer, N. (2022). *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.* JMLR 2022.

Key contribution: Proved that $k = 1$ (single expert per token) is sufficient. Simplified routing reduces communication overhead. Introduced:

- **Load balancing loss:** $\mathcal{L}_{\text{aux}} = \alpha \cdot E \cdot \sum_i f_i \cdot P_i$.
- **Expert capacity:** $C = \lceil \frac{B}{E} \cdot \text{CF} \rceil$, tokens exceeding capacity are dropped.
- **Selective precision:** Use FP32 for router, BF16 for FFN.

10.6 GShard (Lepikhin et al., 2021)

Paper: Lepikhin, D., Lee, H., Xu, Y., *et al.* (2021). *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding.* ICLR 2021.

Key contribution: First production-scale MoE (600B total, used in Google Translate). Introduced:

- **Expert parallelism:** Each expert on a different device. All-to-all communication.
- Random routing for exploration.
- Auxiliary loss for load balancing.

10.7 MoE Design Space

Table 10.1: MoE design decisions across models

Model	**Total**	**Active**	E	k	**Key Feature**
Switch Transf.	1.6T	∼200B	2048	1	Single expert routing
GShard	600B	∼100B	2048	2	Random routing baseline
Mixtral	46.7B	12.9B	8	2	No aux loss, interleaved dense
DBRX	132B	36B	16	4	MegaBlocks, no token drop
DeepSeek-V2	236B	21B	160	6	Shared+routed, fine-grained
DeepSeek-V3	671B	37B	256	8	Aux-loss-free, MTP
Qwen2-MoE	57B	14B	64	8	Upcycled, shared experts
Jamba	52B	12B	16	2	Hybrid Mamba+Attn+MoE
Grok-1	314B	∼78B	8	2	25% layers MoE, open

Chapter 11

State Space Models and Linear-Complexity Alternatives

This chapter covers non-transformer architectures that challenge the attention paradigm.

11.1 Mamba (Gu & Dao, December 2023)

11.1.1 The Paper

Paper: Gu, A., & Dao, T. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.* arXiv:2312.00752.

11.1.2 The Core Problem: Attention is Quadratic

Transformer attention costs $O(L^2)$ in sequence length L. SSMs cost $O(L)$ but prior SSMs (S4, S5) were **content-agnostic**—they processed every token identically because their parameters $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \Delta)$ were time-invariant.

11.1.3 Mamba's Innovation: Selective SSM

Mamba makes the SSM parameters **input-dependent**:

$$\mathbf{B}_t = \text{Linear}_B(\mathbf{x}_t) \quad \text{(input projection learns what to store)} \tag{11.1}$$

$$\mathbf{C}_t = \text{Linear}_C(\mathbf{x}_t) \quad \text{(input projection learns what to output)} \tag{11.2}$$

$$\Delta_t = \text{softplus}(\text{Linear}_\Delta(\mathbf{x}_t) + \text{Bias}_\Delta) \tag{11.3}$$

The state matrix $\mathbf{A}$ remains time-invariant (simplified to a scalar $\times$ identity for efficiency). This selectivity means the model can **selectively remember or ignore** information based on content.

11.1.4 Hardware-Aware Algorithm

The challenge: selectivity breaks the convolutional view (no fixed kernel anymore). Mamba solves this with:

- **Parallel scan (prefix sum):** The SSM recurrence $\mathbf{x}_t = \overline{\mathbf{A}}_t \mathbf{x}_{t-1} + \overline{\mathbf{B}}_t \mathbf{u}_t$ is computed via a parallel prefix scan algorithm, implemented as a custom CUDA kernel.

- **Kernel fusion:** The scan is fused with other operations (projections, activations) into a single GPU kernel to minimize memory traffic.
- **Memory efficiency:** No attention matrix materialization. State size $N = 16$ per channel, expanded to $\sim$2–4$\times$ hidden dim through channel expansion factor.

11.1.5 Architecture Details

- Mamba blocks alternate like transformer blocks.
- Each block: input $\rightarrow$ linear proj $\rightarrow$ 1D conv $\rightarrow$ SiLU $\rightarrow$ SSM $\rightarrow$ residual + output proj.
- **Model sizes:** 130M, 370M, 790M, 1.4B, 2.8B parameters.
- Mamba-2.8B matches transformer models at 2$\times$ the parameter count.

11.1.6 Mamba-2 (Dao & Gu, May 2024)

Paper: Dao, T., & Gu, A. (2024). *Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality.*

Key improvements:

- Proved SSM-attention duality: SSMs = structured (semi-separable) matrices.
- Reformulated as block-decomposed matrix multiplication for tensor core optimization $\rightarrow$ 2–8$\times$ faster than Mamba-1.
- Scalar-times-identity $\mathbf{A}$ for simplicity.
- Larger state dimension ($N = 64$–256).
- Can interleave with attention blocks freely (hybrid architectures).

11.2 RWKV: Receptance Weighted Key Value (Peng et al., 2023)

11.2.1 The Paper

Paper: Peng, B., Alcaide, E., Anthony, Q., *et al.* (2023). *RWKV: Reinventing RNNs for the Transformer Era.* EMNLP 2023.

11.2.2 Architecture

RWKV blends RNN-like recurrence with transformer-like parallel training:

$$\mathbf{r}_t = \mathbf{W}_r \cdot (\mu_r \odot \mathbf{x}_t + (1 - \mu_r) \odot \mathbf{x}_{t-1}) \quad \text{(receptance)} \tag{11.4}$$

$$\mathbf{k}_t = \mathbf{W}_k \cdot \mathbf{x}_t \quad \text{(key)} \tag{11.5}$$

$$\mathbf{v}_t = \mathbf{W}_v \cdot \mathbf{x}_t \quad \text{(value)} \tag{11.6}$$

$$\mathbf{wkv}_t = \frac{\sum_{i=1}^{t-1} e^{-(t-1-i)w+\mathbf{k}_i} \odot \mathbf{v}_i + e^{u+\mathbf{k}_t} \odot \mathbf{v}_t}{\sum_{i=1}^{t-1} e^{-(t-1-i)w+\mathbf{k}_i} + e^{u+\mathbf{k}_t}} \tag{11.7}$$

where w is a learned per-channel decay, u is a "first token bonus" (compensates for the lack of history for token 1). The μ parameters linearly interpolate between current and previous token values.

11.2.3 Training = Parallel, Inference = RNN

Training: the WKV computation can be vectorized (quadratic interaction but parallel over tokens). Inference: only $O(1)$ state update per token (recurrent). This is the core advantage.

11.2.4 RWKV Evolution

- **RWKV-v4 (2023):** The base architecture described above. 14B variant.
- **RWKV-v5 Eagle (2024):** Multi-headed WKV, data-dependent time decay $w_t = f(\mathbf{x}_t)$, improved token shifting with learned exponential decay. Higher quality.
- **RWKV-v6 Finch (2024):** Dynamic token mixing ratio (the μ parameter becomes data-dependent). Further improved quality. 7B model competitive with Mistral 7B.
- **RWKV-7 (Goose, 2025):** Context-dependent gating, dynamic state expansion. Rapidly closing the gap with transformers.

11.3 Other Linear-Complexity Models

11.3.1 Griffin (DeepMind, March 2024)

Paper: De, S., Smith, S.L., Fernando, A., *et al.* (2024). *Griffin: Mixing Gated Linear Recurrences with Local Attention for Efficient Language Models.*

Architecture: RG-LRU (Real-Gated Linear Recurrent Unit):

$$\mathbf{h}_t = \mathbf{a}_t \odot \mathbf{h}_{t-1} + \sqrt{1 - \mathbf{a}_t^2} \odot (\mathbf{b}_t \mathbf{x}_t) \tag{11.8}$$

where $\mathbf{a}_t = \sigma(\mathbf{W}_a \mathbf{x}_t)$ is a data-dependent gate. Combined with sliding window attention. Matches Mamba quality. Used in RecurrentGemma.

11.3.2 Hyena (Poli et al., 2023)

Long convolution parameterized by MLP: $\mathbf{K}[t] = \text{MLP}_{\text{proj}}(\text{MLP}_{\text{body}}(t/L))$. Applied via FFT. Sub-quadratic. Predecessor to Mamba in spirit (implicit state through convolutions).

11.3.3 H3 (Fu et al., 2023)

Stacks two SSMs with multiplicative interaction to emulate attention. Showed that SSMs can approximate the quadratic attention operator.

11.3.4 xLSTM (Beck et al., 2024)

Revives the LSTM with exponential gating ($e^{\mathbf{f}_t}$, $e^{\mathbf{i}_t}$) and matrix memory. Matches or exceeds transformers and Mamba at 1.3B-scale comparisons. Shows LSTMs can be competitive when modernized.

11.3.5 Falcon-Mamba 7B (TII, 2024)

A pure Mamba model at 7B scale trained on 5.5T tokens by the Technology Innovation Institute. Released under Apache 2.0. Demonstrates that SSM-only models can be competitive at open-weight LLM scales. Matches Mistral 7B on many benchmarks.

11.4 The Alternative Architecture Landscape

Table 11.1: Non-transformer architecture comparison

Model	**Complexity**	**Content-Aware**	**Max Scale**	**Key Mechanism**
Mamba-2	$O(L)$	Yes	2.8B+	Selective SSM + parallel scan
RWKV-v6	$O(L)$	Yes (partial)	14B	Exponential decay + token mixing
Griffin	$O(L)$	Yes	14B	RG-LRU gating
xLSTM	$O(L)$	Yes	1.3B	Exponential LSTM gating
Jamba	$O(L)$	Hybrid	52B MoE	Mamba + Attention + MoE
Falcon-Mamba	$O(L)$	Yes	7B	Pure Mamba

Chapter 12

Code-Specialized LLMs

Code generation models have specialized architectures and training recipes distinct from general-purpose LLMs.

12.1 CodeLlama (Meta, August 2023)

12.1.1 The Paper

Paper: Rozière, B., Gehring, J., Gloeckle, F., *et al.* (2023). *Code Llama: Open Foundation Models for Code.* arXiv:2308.12950.

12.1.2 Approach

Initialization: Start from LLaMA 2 foundation models (7B, 13B, 34B). Continue pretraining on code-heavy data. This is the dominant paradigm for code models—bootstrap from a general LLM.

12.1.3 Specialized Training

- **Code Infilling (Fill-in-the-Middle):** Split training samples into prefix, middle, suffix. Train to predict the middle given both context:

```
<PRE> def fibonacci(n): <SUF> return fib <MID>
    if n <= 1: return n
    a, b = 0, 1
    for _ in range(n-1):
        a, b = b, a+b
    return b
```

- **Long context fine-tuning:** Extended context from 4K to 16K via RoPE frequency scaling (NTK-aware interpolation).
- **Python specialization:** A CodeLlama-Python variant further fine-tuned on Python-only data (100B tokens). Provides additional gains on Python tasks.
- **Instruction variant:** CodeLlama-Instruct trained on human-annotated code instruction data + self-instruct generated data.

12.1.4 Results

- CodeLlama-34B: 53.7% on HumanEval (close to GPT-3.5 at 48.1%).
- CodeLlama-Python-34B: 58.5%.
- Showed that multi-lingual code fine-tuning and FIM training are critical.

12.2 StarCoder (BigCode, May 2023–Feb 2024)

12.2.1 StarCoder 1 (BigCode, May 2023)

Paper: Li, R., Allal, L.B., Zi, Y., *et al.* (2023). *StarCoder: May the Source Be With You!*

Training data: The Stack v1.5—6.4 TB of permissively licensed code from GitHub across 358 languages.

Architecture: 15.5B SantaCoder-style architecture (based on GPT-2 but with MQA, FIM training):

- Multi-Query Attention (MQA) for efficiency.
- Fill-in-the-Middle training from the start.
- **8K context.**
- Trained on 1T tokens from The Stack.

12.2.2 StarCoder 2 (BigCode + ServiceNow, Feb 2024)

Paper: Lozhkov, A., Li, R., Allal, L.B., *et al.* (2024). *StarCoder 2 and The Stack v2: The Next Generation.*

Key improvements:

- **The Stack v2:** 16.7 TB, 619 languages. Much larger and more carefully curated.
- **Three sizes:** 3B, 7B, 15B. All use GQA, RoPE, 16K context.
- **Training:** 3B model trained on 3T tokens, 15B on 4T tokens. Over-trained (beyond Chinchilla) for inference efficiency.
- **Repo-level context:** Training includes adjacent files from the same repository to enable multi-file code understanding.
- **Result:** StarCoder2-15B outperforms CodeLlama-34B on HumanEval while being 2× smaller.

12.3 DeepSeek-Coder Series

12.3.1 DeepSeek-Coder V1 (November 2023)

Paper: Guo, D., Zhu, Q., Yang, D., *et al.* (2024). *DeepSeek-Coder: When the Large Language Model Meets Programming—The Rise of Code Intelligence.*

- Sizes: 1.3B, 5.7B, 6.7B, 33B.
- **Trained from scratch on 2T code tokens** (not continued from general LLM). 87% code, 10% code-related NL, 3% general NL.
- **Repo-level pre-training:** Uses dependency graphs to group related files, enabling cross-file understanding.
- **FIM training** from the start with PSM (Prefix-Suffix-Middle) format.
- **16K context** via extended RoPE.

- DeepSeek-Coder-33B: 79.3% on HumanEval (state-of-the-art for open models at release).

12.3.2 DeepSeek-Coder V2 (June 2024)

Based on DeepSeek-V2 architecture (MLA + DeepSeekMoE). 236B total, 21B active. 338 languages. Matched GPT-4 Turbo on code tasks. Open-source.

12.4 Other Notable Code Models

12.4.1 WizardCoder (Luo et al., 2023)

Fine-tuned StarCoder with Evol-Instruct (iterative instruction complexity increase). Achieved SOTA on HumanEval among open models at the time.

12.4.2 CodeQwen 1.5 (Alibaba, 2024)

Based on Qwen 1.5 foundation. GQA, 64K context. Specialized code FIM training. Strong performance on multi-lingual code benchmarks.

12.4.3 Magicoder (Wei et al., 2024)

OSS-Instruct: Use open-source code snippets as seed to generate diverse coding instructions. Fine-tuned CodeLlama. Strong data-efficient approach.

12.4.4 CodeGemma (Google, 2024)

Based on Gemma architecture. Two variants: 2B (base + instruct) and 7B (instruct only). Trained on 500B additional code tokens. Fill-in-the-middle.

12.5 Code Model Design Patterns

Table 12.1: Code model design decisions

Model	Base	Size	FIM	Repo Ctx	HumanEval
CodeLlama	LLaMA 2	7–34B	Yes	No	53.7% (34B)
StarCoder 2	From scratch	3–15B	Yes	Yes	46.3% (15B)
DeepSeek-Coder v1	From scratch	1.3–33B	Yes	Yes	79.3% (33B)
DeepSeek-Coder v2	DeepSeek-V2	236B/21B act.	Yes	Yes	90.2%
CodeGemma	Gemma	2B/7B	Yes	No	55.5% (7B)
WizardCoder	StarCoder	15B	No	No	57.3%

Chapter 13

Community and Open-Source LLM Milestones

Beyond the major labs, open-source communities have produced important models that shaped the ecosystem.

13.1 Falcon (TII, UAE, 2023)

13.1.1 Falcon-40B and 180B (June–November 2023)

Paper: Almazrouei, E., Alobeidli, H., Alshamsi, A., *et al.* (2023). *The Falcon Series of Open Language Models.*

Architecture: Modified decoder-only transformer.

- **Flash Attention + Multi-Query Attention (MQA):** One of the first major models to combine MQA with Flash Attention.
- **Parallel residual streams:** Attention and FFN computed from separate layer norms on the same input (like PaLM).
- **Training data:** RefinedWeb—5T tokens of filtered web data. Quality filtering was a major contribution.
- Falcon-180B was the largest open model at release.

Falcon-Mamba 7B (2024): A pure Mamba SSM model. Apache 2.0 licensed.

13.2 MPT (MosaicML, May 2023)

13.2.1 The Paper

Paper: MosaicML NLP Team (2023). *MPT-7B: A New Standard for Open-Source, Commercially Usable LLMs.*

13.2.2 Architecture

- **Modified decoder-only** with architectural innovations from multiple sources.
- **ALiBi (Attention with Linear Biases, Press et al., 2022):** Instead of RoPE or learned positions, ALiBi adds a static, non-learned bias to attention scores based

on relative distance:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{m} \cdot B\right)\mathbf{V} \tag{13.1}$$

where $B_{ij} = -|i - j|$ and $\mathbf{m}$ is a head-specific slope. This enables extrapolation to longer sequences than those seen during training.

- **No bias terms:** Following LLaMA's approach.
- **Flash Attention, FSDP training.**
- **Sizes:** 7B (MPT-7B). Later MPT-30B.

13.2.3 Context Extension

MPT-7B-StoryWriter-65k+: Fine-tuned with ALiBi to extrapolate to 65K tokens—unprecedented at the time.

Training cost: ∼200K on MosaicML platform over 9.5 days. Demonstrated cost-effective LLM training for enterprises.

13.3 Yi (01.AI, November 2023)

13.3.1 The Paper

Paper: Young, A., Chen, B., Li, C., *et al.* (01.AI, 2024). *Yi: Open Foundation Models by 01.AI.*

13.3.2 Architecture

- Standard LLaMA-like design: RMSNorm, SwiGLU, RoPE, GQA.
- **Sizes:** 6B, 9B, 34B.
- **Vocabulary:** 64K tokens.
- **Training data:** 3.1T tokens. English + Chinese.
- **Dual-stage training:** Stage 1 (2.4T): standard pretraining. Stage 2 (0.7T): data with higher proportion of STEM, logic, reasoning.

13.3.3 Results

Yi-34B outperformed LLaMA-2 70B on several benchmarks. Yi-6B strong for its size. Yi-VL (visual-language) and Yi-Coder extensions followed.

13.4 OLMo (AI2, February 2024)

13.4.1 The Paper

Paper: Groeneveld, D., Beltagy, I., Walsh, P., *et al.* (2024). *OLMo: Accelerating the Science of Language Models.* ACL 2024.

13.4.2 Full Transparency

OLMo is the most transparent LLM release:

- **Open:** Model weights, training data (Dolma dataset), training code, logs, intermediate checkpoints (every 1000 steps).
- **Dolma dataset:** 3T tokens of openly available data. C4, PeS2o, Reddit, Wikipedia, books. Fully documented.
- **Architecture:** LLaMA-like, 7B (with 1B variant). SwiGLU, RoPE, no biases.
- **Unique: Max update parameterization (μP):** Uses μP (Yang et al., 2022) which ensures the optimal learning rate is independent of model width. This allowed AI2 to tune hyperparameters on a small model and transfer them directly to the 7B model without re-tuning.
- **Training logs:** 500 checkpoints, full wandb logs. Unprecedented transparency for reproducibility.

OLMo 2 (December 2024): Improved with 2-stage training. OLMoE: MoE variant with 7B active, 1B expert routing.

13.5 Command R / Command R+ (Cohere, 2024)

13.5.1 Architecture

- **Command R:** 35B dense. **Command R+:** 104B, likely MoE.
- **Optimized for RAG and tool use:** Pre-trained with a mixture that includes retrieval-augmented training examples.
- **Multi-step tool use:** Models are trained to iteratively call tools, analyze results, and refine actions over multiple turns.
- **128K context.**
- **Multilingual:** 23 languages.
- Unique: Command R+ is optimized for enterprise use (grounded generation, citation of sources).

13.6 BLOOM (BigScience, July 2022)

13.6.1 The Paper

Paper: Scao, T.L., Fan, A., Akiki, C., *et al.* (2022). *BLOOM: A 176B-Parameter Open-Access Multilingual Language Model.*

13.6.2 Architecture

- 176B parameters. **Decoder-only** architecture.
- **ALiBi** positional embeddings (like MPT, not RoPE).
- **LayerNorm** (not RMSNorm).
- **GELU activation** (not SwiGLU).
- **Vocabulary:** 250K tokens (46 languages).

BLOOM was historically significant as the first 100B+ open-access multilingual model, though its older architecture choices made it less efficient than LLaMA-era models.

13.7 OPT (Meta, May 2022)

13.7.1 The Paper

Paper: Zhang, S., Roller, S., Goyal, N., *et al.* (2022). *OPT: Open Pre-trained Transformer Language Models.*

13.7.2 Architecture

- Replicates GPT-3's architecture (dense decoder, learned position embeddings).
- Sizes: 125M to 175B.
- **Historical importance:** First open replication of GPT-3 at full scale. Enabled research on large model behaviors without API access.
- Used ReLU activation (not SwiGLU). Pre-LayerNorm architecture.

13.8 Amazon: Titan, Olympus, Nova

- **Titan (2023):** AWS Bedrock models. Limited architecture details.
- **Olympus (2024):** Rumored 2T parameter model. Cancelled or not released.
- **Nova (December 2024):** Multimodal foundation models. Nova Micro, Lite, Pro, Premier. Focused on AWS cloud deployment.

13.9 Other Notable Models

- **Skywork (2023–2024):** Chinese-English models from Kunlun Tech. 13B, competitive with Qwen/Yi.
- **Baichuan (2023):** Chinese-focused LLMs from Baichuan AI. 7B, 13B. MoE variants (Baichuan-M1).
- **Hunyuan (Tencent, 2024):** 389B MoE (52B active). Competitive with DeepSeek-V2.
- **Ernie (Baidu):** One of the earliest Chinese LLMs. 4.0 version claims GPT-4 parity.
- **MiniCPM (2024):** Small-scale (2B) model competitive with 7B models via distillation from larger models. Phone-deployable.
- **Aya (Cohere, 2024):** Multilingual model covering 101 languages. 8B and 35B variants.
- **StableLM (Stability AI, 2023):** 3B model. Zephyr 7B (fine-tuned Mistral) outperformed many larger models through DPO.
- **Tulu 2 (Allen AI, 2023):** Demonstrated that careful data mixing is more important than model size for instruction following.

Chapter 14

Instruction-Tuned and Preference-Aligned Models

Fine-tuned models demonstrate that post-training is as important as pre-training for real-world usability.

14.1 Vicuna (LMSYS, March 2023)

14.1.1 The Paper

Paper: Chiang, W.-L., Li, Z., Lin, Z., *et al.* (2023). *Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality.*

14.1.2 Approach

- Fine-tuned LLaMA on **ShareGPT conversations** (∼70K user-assistant dialogues scraped from sharegpt.com).
- **Training cost:** ∼300 USD on 8× A100 GPUs. Demonstrated that small-scale fine-tuning can produce competitive chatbots.
- **Multi-turn conversation format:** Trained with full dialogue history.
- **Cosine LR, 3 epochs.**

14.1.3 Significance

Vicuna sparked the "fine-tune LLaMA" ecosystem. It proved that:

- High-quality conversation data matters more than quantity.
- A single A100 node can produce a GPT-3.5-competitive chatbot.
- Multi-turn training is essential for conversational AI.

14.2 WizardLM (Microsoft, April 2023)

14.2.1 The Paper

Paper: Xu, C., Sun, Q., Zheng, K., *et al.* (2023). *WizardLM: Empowering Large Language Models to Follow Complex Instructions.*

14.2.2 Evol-Instruct Method

Iteratively increases instruction complexity:

1. Start with a set of seed instructions.
2. **In-Depth Evolving:** Add constraints, deepen reasoning, increase steps, complicate input.
3. **In-Breadth Evolving:** Generate new topic variations, increase diversity.
4. Use a strong LLM (ChatGPT) to both evolve the instruction AND generate the response.
5. Filter out failed evolutions.
6. Fine-tune on the evolved dataset.

Result: WizardLM-7B (LLaMA fine-tune) outperformed ChatGPT on 10% of test prompts.

14.2.3 WizardCoder / WizardMath

Applied Evol-Instruct to code datasets (WizardCoder) and math datasets (WizardMath), achieving state-of-the-art results for open models on HumanEval and GSM8K.

14.3 Orca (Microsoft, June 2023)

14.3.1 The Paper

Paper: Mukherjee, S., Mitra, A., Jawahar, G., *et al.* (2023). *Orca: Progressive Learning from Complex Explanation Traces of GPT-4.*

14.3.2 Progressive Learning

Orca distills not just outputs but **reasoning traces**:

- For each training example, collect from GPT-4: (a) the **explanation** (step-by-step reasoning), (b) the final answer.
- Train the student model to generate both.
- **System instructions:** GPT-4 provides detailed system messages that explain the task and reasoning approach.
- **Progressive complexity:** Start with simple explanations, gradually increase complexity and reasoning depth.

Orca-2 (2024): Trained on LLaMA-2 13B. Introduces **prompt erasure**—training the model to perform well even when the explanatory system prompt is removed, forcing internalization of reasoning strategies.

14.3.3 Results

Orca-13B matched ChatGPT on several benchmarks. Demonstrated that **step-by-step reasoning transfer** is more effective than simple output distillation.

14.4 Zephyr (HuggingFace, October 2023)

14.4.1 The Paper

Paper: Tunstall, L., Beeching, E., Lambert, N., *et al.* (2023). *Zephyr: Direct Distillation of LM Alignment.*

14.4.2 Three-Stage Alignment

Stage 1—SFT: Fine-tune Mistral 7B on UltraChat (1.4M synthetic dialogues generated by GPT-3.5).

Stage 2—AI Feedback Collection: For each prompt, generate multiple responses. GPT-4 scores each response. The scores become preference data.

Stage 3—DPO (Direct Preference Optimization): Apply DPO to the SFT model using the AI feedback data. No reward model, no PPO:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}[\log \sigma(\beta \log \frac{\pi_\theta(y_w|x)}{\pi_{\text{ref}}(y_w|x)} - \beta \log \frac{\pi_\theta(y_l|x)}{\pi_{\text{ref}}(y_l|x)})] \tag{14.1}$$

14.4.3 Results

Zephyr-7B (β variant) matched LLaMA-2-Chat 70B on MT-Bench. Demonstrated that:

- DPO is simpler and as effective as RLHF PPO.
- AI feedback (GPT-4 as judge) can replace human feedback.
- A 7B model can match 70B models through careful alignment.

Zephyr-141B-A35B (Mixtral-based, 2024): Zephyr alignment applied to Mixtral.

14.5 NeuralChat (Intel, 2023–2024)

Intel's fine-tuning framework emphasizing:

- **Habana Gaudi acceleration:** Optimized for Intel's AI accelerator.
- **Multi-task fine-tuning:** Jointly trained on dialogue, summarization, translation, code.
- **Fast SFT:** Conditional computation SFT for rapid iteration.

14.6 UltraChat / UltraFeedback (2023–2024)

14.6.1 UltraChat (Ding et al., 2023)

1.4M multi-turn dialogues generated by GPT-3.5 with diverse topics and styles. Used as SFT data by Zephyr, Starling, and many other models.

14.6.2 UltraFeedback (Cui et al., 2023)

64K prompts, 256K responses, 1M GPT-4 judgments. Each response scored on: helpfulness, honesty, harmlessness, instruction-following. Became the standard AI feedback dataset.

14.7 Airoboros (2023)

Open-source fine-tuning dataset and model series emphasizing:

- Multi-turn conversations with character/role-play.
- Chain-of-thought reasoning.
- Coding with execution feedback.
- Avoidance of GPT-4-generated data (used Claude/self-generated data).

14.8 Alignment Model Comparison

Table 14.1: Fine-tuning approaches compared

Model	**Base**	**Method**	**Feedback**	**Key Insight**
Vicuna	LLaMA	SFT	None (ShareGPT)	Dialogue data quality
WizardLM	LLaMA	Evol-Instruct SFT	None	Complexity evolution
Orca	LLaMA	Progressive SFT	GPT-4 traces	Reasoning transfer
Zephyr	Mistral	SFT + DPO	GPT-4 scores	DPO beats RLHF PPO
Starling	Mistral	SFT + RLHF	GPT-4 + human	Iterative refinement

Chapter 15

Training Paradigms and Scaling Innovations

This chapter covers the foundational ideas about how to train LLMs efficiently and effectively.

15.1 The Scaling Laws (Kaplan et al., 2020)

15.1.1 The Paper

Paper: Kaplan, J., McCandlish, S., Henighan, T., *et al.* (2020). *Scaling Laws for Neural Language Models.*

15.1.2 Key Findings

Given a compute budget C (in FLOPs), the optimal model size N and training tokens D follow:

$$N_{\text{opt}}(C) \propto C^{0.73} \quad \text{(model size dominates)} \tag{15.1}$$

$$D_{\text{opt}}(C) \propto C^{0.27} \quad \text{(data much less important)} \tag{15.2}$$

This implied that for a $10\times$ compute increase, you should increase model size by $5.6\times$ and data by only $1.8\times$. This led to the "big models, not big data" era.

The power-law: Cross-entropy loss L follows:

$$L(N, D) = \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} + L_\infty \tag{15.3}$$

where $N_c, D_c, \alpha_N, \alpha_D, L_\infty$ are fitted constants.

Learning rate transfer: The optimal learning rate is largely independent of model size (when using the right parameterization). Larger models need larger batch sizes but similar LR.

15.2 Chinchilla: Compute-Optimal Training (Hoffmann et al., 2022)

15.2.1 Revision of Kaplan's Laws

Chinchilla found that Kaplan et al. **underestimated the importance of data**. The corrected scaling:

$$N_{\text{opt}}(C) \approx C^{0.49} \quad \text{(parameters and tokens)} \tag{15.4}$$
$$D_{\text{opt}}(C) \approx C^{0.51} \quad \text{(should grow at the same rate!)} \tag{15.5}$$

The optimal ratio: $\sim$20 tokens per parameter. This upended the field.

15.2.2 Why Kaplan's Laws Were Wrong

Three methodological differences:

1. Kaplan analyzed the final loss separately for each model size; Chinchilla jointly fit a parametric loss function.
2. Kaplan used a fixed LR schedule; Chinchilla used cosine decay (which requires knowing the total steps).
3. Kaplan analyzed only in the "compute-limited" regime (empirically, large models were under-trained).

15.3 Maximal Update Parameterization (μP, Yang et al., 2022)

15.3.1 The Paper

Paper: Yang, G., Hu, E., Babuschkin, I., *et al.* (2022). *Tensor Programs V: Tuning Large Neural Networks via Zero-Shot Hyperparameter Transfer.*

15.3.2 The Problem

Hyperparameter tuning at scale is expensive. The optimal learning rate, initialization scale, etc., change with model size in non-obvious ways.

15.3.3 The Solution: μP

μTransfer provides rules for how to scale initialization, LR, and multipliers with width so that the optimal hyperparameters for a small model transfer directly to a large model:

- **Initialization:** Weights $\sim \mathcal{N}(0, 1/fan_in)$, scaled per layer type differently.
- **Learning rate:** Embedding LR $\propto 1$, hidden LR $\propto 1/width$, output LR $\propto 1/width$.
- **Multiplier scaling:** Residual connections, attention logit scaling all follow width-dependent rules.

Used by: OLMo (AI2), Cerebras-GPT, and increasingly by frontier labs.

15.4 Data Curation and Filtering Innovations

15.4.1 Deduplication

- **MinHash LSH:** Approximate deduplication at web scale. Hash each document into signatures, find near-duplicates via LSH.
- **Exact substring dedup:** Remove exact duplicate spans (suffix array-based). Standard in LLaMA pipeline.
- **Semantic dedup:** Embed documents and cluster to find semantically duplicate content.

15.4.2 Quality Filtering

- **Perplexity-based:** Train a small LM on high-quality data; score candidate data. Keep only data in a "perplexity band" (not too random, not too repetitive).
- **Classifier-based:** Train a fastText/KENLM classifier to distinguish high-quality text (Wikipedia, books) from low-quality (comments, spam). Used by LLaMA, C4.
- **Educational value:** Phi's approach—score each example for "how much can a learner learn from this text."

15.4.3 Data Mixing and Curriculum

- **Dynamic mixing:** Change the data mixture during training. LLaMA-3 uses 3 stages with increasing code/math proportion.
- **DoReMi (Domain Reweighting with Minimax Optimization, Xie et al., 2023):** Automatically learns optimal domain weights by training a small proxy model to minimize worst-case excess loss across domains.
- **Annealing:** In the final 10–20% of training, reduce LR and switch to higher-quality data ("cooldown" phase).

15.5 Training Infrastructure Innovations

15.5.1 3D Parallelism (Megatron-LM, DeepSpeed)

- **TP (Tensor Parallelism):** Split individual weight matrices across GPUs within a node (uses NVLink).
- **PP (Pipeline Parallelism):** Split layers across GPUs/nodes.
- **DP (Data Parallelism):** Replicate model, different data per replica.
- **ZeRO (Rajbhandari et al., 2020):** Partition optimizer states (Stage 1), gradients (Stage 2), and parameters (Stage 3) across DP replicas. Enables training 100B+ models on commodity hardware.

15.5.2 FSDP (Fully Sharded Data Parallel, PyTorch)

PyTorch's native version of ZeRO-3. Params, gradients, optimizer states all sharded across GPUs. All-gathered before each layer's forward, discarded after backward. Integrated with `torch.compile` for compilation.

15.5.3 Mixed Precision Training

- **BF16:** Google Brain Float 16. Same exponent range as FP32 (8 exponent bits), 7 mantissa bits. Used by most modern LLMs.
- **FP8 training (DeepSeek-V3):** E4M3 format. GEMM in FP8, attention/norm in BF16. Block-wise quantization.
- **Automatic Mixed Precision (AMP):** Framework automatically selects which operations can run in lower precision.

15.5.4 Activation Checkpointing / Gradient Checkpointing

Trade compute for memory: during forward pass, only store activations at a subset of layers (checkpoints). During backward pass, recompute intermediate activations from the nearest checkpoint. Reduces peak memory by $\sim O(\sqrt{L})$ for an $O(L)$ increase in compute.

15.5.5 Sequence Packing

Pack multiple short documents into one 4096/8192-token sequence (with special separator tokens). Avoids wasted computation on padding. Standard in all modern LLM training (LLaMA, Mistral, DeepSeek).

15.6 Beyond Next-Token Prediction

15.6.1 Multi-Token Prediction (Gloeckle et al., 2024; DeepSeek-V3)

Predict n future tokens simultaneously. DeepSeek-V3 uses $n = 2$ with additional prediction heads. Benefits: richer training signal, improved sample efficiency, enables speculative decoding at inference.

15.6.2 UL2 (Tay et al., 2022)—Mixture of Denoisers

A unified pre-training objective mixing different denoising tasks:

- **S-denoising (sequential):** Standard causal LM (like GPT).
- **R-denoising (regular):** Span corruption like T5 (mask-then-predict).
- **X-denoising (extreme):** Very long span corruption or extreme compression.

These are mixed with a mode-switching token. UL2 demonstrated that diverse pre-training objectives improve downstream performance.

15.6.3 FIM (Fill-in-the-Middle)

Standard for code models. PSM format: `<PRE>` prefix `<SUF>` suffix `<MID>` middle `<EOS>`. Trains bidirectional context use within an autoregressive framework.

15.6.4 Knowledge Distillation in Pre-Training

Some models (Gemma, Phi) incorporate distillation from larger teachers during pre-training:

- **Token-level distillation:** Student matches teacher's next-token probability distribution.
- **Hidden-state distillation:** Match intermediate representations.
- **Multi-teacher:** Distill from multiple teacher models simultaneously.

15.7 Training Efficiency Summary

Table 15.1: Training innovations across models

Technique	Introduced By	Benefit	Adopted By
BF16 training	Google (2018)	2× memory, stability	All modern LLMs
Chinchilla scaling	DeepMind (2022)	Optimal data/param ratio	LLaMA, DeepSeek
FSDP / ZeRO-3	Meta / MS (2020)	Train 175B on 128 GPUs	LLaMA, OLMo
Flash Attention	Dao et al. (2022)	2–4× faster attention	All modern LLMs
μP	Yang et al. (2022)	Zero-shot HP transfer	OLMo, Cerebras
FP8 training	NVIDIA / DeepSeek	2× faster, 40% less mem	DeepSeek-V3
MTP	DeepSeek (2024)	Sample efficiency, spec. dec.	DeepSeek-V3
Sequence packing	Various	No padding waste	All modern LLMs
Cooldown/annealing	LLaMA-3	Final quality boost	LLaMA-3, Qwen2.5

Afterword

From GPT-1's 117M parameters to DeepSeek-V3's 671B MoE, the evolution of LLM architectures represents one of the most rapid technological progressions in history. Each model family contributed critical innovations—attention mechanisms, scaling laws, efficient architectures, alignment techniques, and deployment strategies—that compound to enable today's capabilities. Understanding this lineage is not just academic: it reveals the design patterns that will shape the next generation of models.